

# Foundations of Control Theory and Engineering

Course Team

October 22, 2025



# Contents

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| <b>1</b> | <b>Orientation and Mathematical Refresh</b>                | <b>9</b>  |
| 1.1      | Concept Map for Beginners . . . . .                        | 9         |
| 1.2      | Concept–Math–Engineering Loop . . . . .                    | 9         |
| 1.3      | Why Study Control? . . . . .                               | 9         |
| 1.4      | Using This Course . . . . .                                | 10        |
| 1.4.1    | Document Workflow . . . . .                                | 11        |
| 1.4.2    | Python Companion Environment . . . . .                     | 11        |
| 1.5      | Mathematical Toolkit . . . . .                             | 11        |
| 1.5.1    | Single-Variable Calculus . . . . .                         | 12        |
| 1.5.2    | Jacobian Matrices . . . . .                                | 12        |
| 1.5.3    | Ordinary Differential Equations . . . . .                  | 13        |
| 1.5.4    | Linear Algebra Refresher . . . . .                         | 15        |
| 1.5.5    | Complex Exponentials and Transforms . . . . .              | 16        |
| 1.6      | Learning Roadmap . . . . .                                 | 16        |
| 1.7      | Keyword Review . . . . .                                   | 17        |
| <b>2</b> | <b>Modeling Dynamic Systems</b>                            | <b>19</b> |
| 2.1      | Concept Map for Beginners . . . . .                        | 19        |
| 2.2      | Concept–Math–Engineering Loop . . . . .                    | 19        |
| 2.3      | Mathematical Background and Modeling Assumptions . . . . . | 19        |
| 2.4      | Mass–Spring–Damper Derivation . . . . .                    | 21        |
| 2.4.1    | Step 1: Coordinate and Force Definition . . . . .          | 21        |
| 2.4.2    | Step 2: First-Order State Representation . . . . .         | 21        |
| 2.4.3    | Step 3: Energy Cross-Check . . . . .                       | 21        |
| 2.4.4    | Final Formulas and Code Mapping . . . . .                  | 22        |
| 2.4.5    | Numerical Validation . . . . .                             | 22        |
| 2.5      | Inverted Pendulum Linearization . . . . .                  | 22        |
| 2.5.1    | Step 1: Nonlinear Dynamics . . . . .                       | 23        |
| 2.5.2    | Step 2: Linearization About Upright Equilibrium . . . . .  | 23        |
| 2.5.3    | Step 3: Code Mapping . . . . .                             | 23        |
| 2.6      | Transfer-Function Connection . . . . .                     | 23        |
| 2.6.1    | Step-by-Step Derivation . . . . .                          | 24        |
| 2.6.2    | Summary . . . . .                                          | 24        |
| 2.7      | Zero-Order Hold Discretization . . . . .                   | 24        |
| 2.7.1    | Derivation . . . . .                                       | 25        |
| 2.7.2    | Numerical Example . . . . .                                | 25        |
| 2.7.3    | Code Mapping . . . . .                                     | 25        |

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| 2.8      | Model Validation, Identification, and Refinement | 25        |
| 2.9      | Hands-On Practice                                | 26        |
| 2.10     | Keyword Review                                   | 27        |
| <b>3</b> | <b>Time-Domain Analysis</b>                      | <b>29</b> |
| 3.1      | Concept Map for Beginners                        | 29        |
| 3.2      | Concept–Math–Engineering Loop                    | 29        |
| 3.3      | Stability by Eigenvalues and Lyapunov Functions  | 29        |
| 3.3.1    | Step-by-Step Derivation                          | 30        |
| 3.3.2    | Code Mapping                                     | 30        |
| 3.4      | Characteristic Polynomials and Routh–Hurwitz     | 31        |
| 3.4.1    | Derivation Workflow                              | 31        |
| 3.4.2    | Code Mapping                                     | 31        |
| 3.5      | Second-Order Response Metrics                    | 31        |
| 3.5.1    | Derivation                                       | 32        |
| 3.5.2    | Worked Specification Example                     | 32        |
| 3.5.3    | Code Mapping                                     | 32        |
| 3.6      | Root-Locus Construction                          | 32        |
| 3.6.1    | Manual Derivation                                | 33        |
| 3.6.2    | Example                                          | 33        |
| 3.6.3    | Code Mapping                                     | 33        |
| 3.7      | PID Controllers: Time-Domain Interpretation      | 33        |
| 3.7.1    | Incremental Implementation                       | 34        |
| 3.7.2    | Simulation Case Study                            | 34        |
| 3.7.3    | Code Mapping                                     | 34        |
| 3.8      | Disturbance and Noise Rejection                  | 35        |
| 3.8.1    | Derivation                                       | 35        |
| 3.8.2    | Case Study                                       | 35        |
| 3.9      | Workflow Summary and Practice                    | 35        |
| 3.10     | Module Review                                    | 37        |
| 3.11     | Keyword Review                                   | 37        |
| <b>4</b> | <b>Frequency-Domain Foundations</b>              | <b>39</b> |
| 4.1      | Concept Map for Beginners                        | 39        |
| 4.2      | Concept–Math–Engineering Loop                    | 39        |
| 4.3      | Sinusoidal Steady-State Response                 | 39        |
| 4.3.1    | Derivation                                       | 40        |
| 4.3.2    | Code Mapping                                     | 40        |
| 4.4      | Bode Plot Construction                           | 40        |
| 4.4.1    | Analytic Template                                | 41        |
| 4.4.2    | Worked Example: Mass–Spring–Damper               | 41        |
| 4.4.3    | Python Reproduction                              | 41        |
| 4.5      | Nyquist Criterion and Stability Margins          | 42        |
| 4.5.1    | Derivation                                       | 42        |
| 4.5.2    | Case Study: PID-Regulated MSD                    | 42        |
| 4.5.3    | Code Mapping                                     | 43        |
| 4.6      | Sensitivity, Nichols, and Loop-Shaping           | 43        |
| 4.6.1    | Key Relations                                    | 43        |

|          |                                               |           |
|----------|-----------------------------------------------|-----------|
| 4.6.2    | Numerical Example                             | 43        |
| 4.6.3    | Code Mapping                                  | 44        |
| 4.7      | Practice Workflow                             | 44        |
| 4.8      | Module Review                                 | 45        |
| 4.9      | Keyword Review                                | 45        |
| <b>5</b> | <b>Classical Controller Design</b>            | <b>49</b> |
| 5.1      | Concept Map for Beginners                     | 49        |
| 5.2      | Concept–Math–Engineering Loop                 | 49        |
| 5.3      | PID Controller Derivation                     | 49        |
| 5.3.1    | Continuous-Time Formula                       | 50        |
| 5.3.2    | Discrete Realization                          | 50        |
| 5.3.3    | Code Mapping                                  | 51        |
| 5.4      | Empirical Tuning Templates                    | 51        |
| 5.4.1    | Ziegler–Nichols                               | 51        |
| 5.4.2    | Cohen–Coon                                    | 51        |
| 5.4.3    | Direct Synthesis                              | 52        |
| 5.5      | Lead/Lag Compensation                         | 52        |
| 5.5.1    | Lead Design Steps                             | 52        |
| 5.5.2    | Lag Design Steps                              | 52        |
| 5.5.3    | Code Mapping                                  | 53        |
| 5.6      | Case Study: Second-Order Position Control     | 53        |
| 5.6.1    | Simulation Verification                       | 53        |
| 5.7      | Practice Workflow                             | 54        |
| 5.8      | Classical and Modern Connections              | 54        |
| 5.9      | Module Review                                 | 56        |
| 5.10     | Keyword Review                                | 56        |
| <b>6</b> | <b>State-Space Methods and Modern Control</b> | <b>57</b> |
| 6.1      | Concept Map for Beginners                     | 57        |
| 6.2      | Concept–Math–Engineering Loop                 | 57        |
| 6.3      | Why State-Space?                              | 57        |
| 6.3.1    | Connecting to Transfer Functions              | 59        |
| 6.3.2    | Worked Example: Mass–Spring–Damper            | 59        |
| 6.4      | Controllability and Observability             | 59        |
| 6.4.1    | Numerical Check                               | 60        |
| 6.5      | Pole Placement                                | 60        |
| 6.6      | Linear Quadratic Regulation                   | 61        |
| 6.6.1    | Servo Prefilter                               | 62        |
| 6.7      | Kalman Filtering in Depth                     | 62        |
| 6.7.1    | Projection Derivation                         | 63        |
| 6.7.2    | Innovation Interpretation                     | 63        |
| 6.7.3    | Steady-State Filter and Riccati Link          | 64        |
| 6.7.4    | Practical Tuning Workflow                     | 64        |
| 6.7.5    | Case Study 1: Position-Only Sensing           | 64        |
| 6.7.6    | Case Study 2: Sensor Fusion with Bias         | 65        |
| 6.7.7    | Case Study 3: Missing Data and Outliers       | 65        |
| 6.8      | Integrated Example: LQR with Kalman Filter    | 65        |

|          |                                                            |           |
|----------|------------------------------------------------------------|-----------|
| 6.9      | Tutorial: LQR and Kalman Practice                          | 66        |
| 6.10     | Design Trade-Offs                                          | 67        |
| 6.11     | Module Review                                              | 68        |
| 6.12     | Keyword Review                                             | 68        |
| <b>7</b> | <b>Estimation and Adaptive Control</b>                     | <b>71</b> |
| 7.1      | Concept Map for Beginners                                  | 71        |
| 7.2      | Concept–Math–Engineering Loop                              | 71        |
| 7.3      | Deterministic State Observers                              | 71        |
| 7.3.1    | Derivation                                                 | 72        |
| 7.3.2    | Design Steps                                               | 72        |
| 7.3.3    | Code Mapping                                               | 72        |
| 7.4      | Kalman Filtering                                           | 72        |
| 7.4.1    | Stochastic Model                                           | 73        |
| 7.4.2    | Tuning Guidelines                                          | 73        |
| 7.4.3    | Code Mapping and Logging                                   | 74        |
| 7.4.4    | Case Study                                                 | 74        |
| 7.5      | Model Reference Adaptive Control                           | 74        |
| 7.5.1    | Derivation                                                 | 74        |
| 7.5.2    | Stability Insight                                          | 75        |
| 7.5.3    | Code Mapping                                               | 75        |
| 7.5.4    | Simulation Practice                                        | 75        |
| 7.6      | Tutorial: Adaptive Control Walkthrough                     | 75        |
| 7.7      | Handling Missing or Delayed Measurements                   | 76        |
| 7.8      | Hands-On Practice                                          | 77        |
| 7.9      | Module Review                                              | 78        |
| 7.10     | Keyword Review                                             | 79        |
| <b>8</b> | <b>Digital Implementation and Practical Considerations</b> | <b>81</b> |
| 8.1      | Concept Map for Beginners                                  | 81        |
| 8.2      | Concept–Math–Engineering Loop                              | 81        |
| 8.3      | Sampling Theory Refresher                                  | 81        |
| 8.3.1    | Worked Example: Aliasing and Timing Jitter                 | 82        |
| 8.4      | Controller Discretization                                  | 83        |
| 8.5      | Quantization, Saturation, and Noise                        | 83        |
| 8.5.1    | Code Mapping                                               | 84        |
| 8.6      | Tutorial: Digital Control Experimentation                  | 84        |
| 8.7      | Scheduling and Real-Time Constraints                       | 86        |
| 8.8      | Validation Pipeline                                        | 87        |
| 8.9      | Implementation Checklist                                   | 87        |
| 8.10     | Module Review                                              | 88        |
| 8.11     | Keyword Review                                             | 88        |
| <b>9</b> | <b>Learning-Based Control and PDE Adaptive Systems</b>     | <b>89</b> |
| 9.1      | Concept Map for Beginners                                  | 89        |
| 9.2      | Concept–Math–Engineering Loop                              | 89        |
| 9.3      | Learning Models from Data                                  | 89        |
| 9.3.1    | Recursive Least Squares (RLS)                              | 90        |

|           |                                                    |            |
|-----------|----------------------------------------------------|------------|
| 9.3.2     | Code Mapping                                       | 90         |
| 9.3.3     | Tutorial: Parameter Identification                 | 90         |
| 9.4       | Stability Theory Refresher                         | 91         |
| 9.5       | Adaptive Control of PDEs                           | 92         |
| 9.5.1     | Code Mapping                                       | 93         |
| 9.5.2     | Tutorial: Adaptive PDE Control                     | 93         |
| 9.6       | Putting It Together                                | 93         |
| 9.7       | Practice Checklist                                 | 94         |
| 9.8       | Module Review                                      | 95         |
| 9.9       | Keyword Review                                     | 95         |
| <b>10</b> | <b>Active Disturbance Rejection Control (ADRC)</b> | <b>97</b>  |
| 10.1      | Concept Map for Beginners                          | 97         |
| 10.2      | Concept–Math–Engineering Loop                      | 97         |
| 10.3      | Architecture Overview                              | 97         |
| 10.4      | Linear ADRC Derivation                             | 98         |
| 10.4.1    | Extended State Observer                            | 99         |
| 10.4.2    | Control Law and Closed-Loop Poles                  | 100        |
| 10.4.3    | Code Mapping                                       | 100        |
| 10.5      | Stability and Disturbance Rejection                | 100        |
| 10.5.1    | Error Dynamics                                     | 101        |
| 10.5.2    | Lyapunov Interpretation                            | 101        |
| 10.6      | Tuning Workflow                                    | 101        |
| 10.7      | Case Study: Mass–Spring–Damper                     | 102        |
| 10.8      | Tutorial: ADRC Walkthrough                         | 102        |
| 10.9      | Discussion and Further Reading                     | 103        |
| 10.10     | Module Review                                      | 104        |
| 10.11     | Keyword Review                                     | 105        |
| <b>A</b>  | <b>Mathematical Background</b>                     | <b>107</b> |
| A.1       | Linear Algebra Reference                           | 107        |
| A.1.1     | Eigenvalues and Modal Decomposition                | 107        |
| A.1.2     | Matrix Norms and Conditioning                      | 107        |
| A.1.3     | Pseudoinverse                                      | 107        |
| A.2       | Differential Equations                             | 107        |
| A.2.1     | Variation of Parameters                            | 107        |
| A.2.2     | Riccati Equations                                  | 108        |
| A.3       | Laplace Transform Reference                        | 108        |
| A.4       | $z$ -Transform Reference                           | 108        |
| A.5       | Probability Primer                                 | 109        |
| A.5.1     | Gaussian Random Vectors                            | 109        |
| A.5.2     | Law of Total Expectation and Variance              | 109        |
| A.6       | Scalar Kalman Filter Recap                         | 109        |
| <b>B</b>  | <b>Tooling and Workflow</b>                        | <b>111</b> |
| B.1       | Repository Layout                                  | 111        |
| B.2       | Compiling the Notes                                | 111        |
| B.3       | Python Environment                                 | 111        |

|                                |            |
|--------------------------------|------------|
| B.4 Automation Hooks . . . . . | 111        |
| <b>References</b>              | <b>113</b> |



# Chapter 1

## Orientation and Mathematical Refresh

### Learning Objectives

- Describe the role of control systems across engineering domains and how this course is organized.
- Refresh the calculus and linear algebra tools repeatedly used in later derivations.
- Configure the computational workflow that pairs LaTeX derivations with Python simulations and plots.

### 1.1 Concept Map for Beginners

#### Concept Map

1. Anchor every new idea to an everyday closed-loop example (thermostats, cruise control) so abstract symbols immediately have a physical story.
2. Revisit the small collection of calculus and linear algebra tools that drive all later derivations, highlighting where they will reappear.
3. Build the code-and-notes workflow upfront so that every formula can be tested with the provided Python scripts minutes after you meet it on paper.

### 1.2 Concept–Math–Engineering Loop

### 1.3 Why Study Control?

#### What?

Control designs feedback laws that steer dynamic systems toward desired behaviours despite disturbances and uncertainty.

#### Why?

Feedback is the glue that holds modern engineering systems together—from autopilots to chemical reactors—so understanding it unlocks broad career pathways.

| Concept                  | Mathematics                                   | Engineering Practice                                                                                                      |
|--------------------------|-----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| What a control loop does | Feedback block diagrams, equilibrium analysis | Sketch block diagrams for a familiar system (home heating) and label the signals.                                         |
| Modeling ingredients     | Taylor series, Jacobians, matrix exponentials | Run <code>mass_spring_damper.linearize()</code> to see symbolic derivatives show up numerically.                          |
| Tooling workflow         | Sets, vectors, reproducibility                | Activate <code>.venv</code> , install requirements, and execute <code>python demo.py pid</code> to connect math to plots. |

Table 1.1: How Module 1 links core concepts, enabling math, and hands-on engineering practice.

### When?

Whenever outputs must track references or respect safety constraints in the presence of changing environments or plant parameters.

### How?

Model the plant, analyse stability and performance, then synthesise controllers that close the loop.

### Example

Cruise control adjusts throttle based on speed error so a car holds 100 km/h up and down hills without driver intervention.

Control theory designs decision-making rules (*controllers*) that regulate dynamic systems so that outputs track desired references despite disturbances and uncertainty. Examples include quadrotor attitude stabilization, cruise control for vehicles, temperature regulation in process plants, and biomedical drug delivery. Throughout the text we will focus on three pillars:

1. **Modeling** — constructing mathematical representations of physical processes (Chapter 2).
2. **Analysis** — predicting stability and performance from the model (Chapters 3 and 4).
3. **Design & Implementation** — synthesizing controllers and estimators, then deploying them digitally (Chapters 5–8).

## 1.4 Using This Course

### What?

A reproducible workflow that unites L<sup>A</sup>T<sub>E</sub>X derivations with Python simulations and figures.

### Why?

Seeing theory and code side by side accelerates learning and mirrors how professional control engineers iterate.

### When?

At the start of each module—set up the tools first so you can verify every formula immediately.

**How?**

Follow the Makefile targets, activate the virtual environment, and run the curated scripts referenced throughout the notes.

**Example**

Compile the book with `make`, then run `python demo.py pid --output figures/pid.png` to generate plots embedded in Chapter 3.

**1.4.1 Document Workflow**

All material is authored in  $\text{\LaTeX}$  and compiled with  $\text{\XeLaTeX}$  using the `Makefile`. Each chapter introduces learning objectives, theoretical development, and explicit references to executable Python examples located in `scripts/`. Appendix B (Tooling and Workflow) collects step-by-step build commands and figure-generation recipes; refer to it whenever you need a quick reminder.

**1.4.2 Python Companion Environment**

Activate the virtual environment and install dependencies with

```
source .venv/bin/activate
pip install -r requirements.txt
```

The command

```
python demo.py pid
```

invokes the canonical mass–spring–damper experiment. By default, plots are displayed using Matplotlib; pass `--no-plot` to suppress figures when running in headless environments and `--output` to save them for inclusion in the notes. Appendix B also tabulates the supported experiments and recommended output directories so you can regenerate figures chapter by chapter.

**1.5 Mathematical Toolkit****What?**

A compact refresher on calculus and linear algebra tools underpinning every derivation in the course.

**Why?**

Mastery of Taylor series, Jacobians, and matrix exponentials turns intimidating control proofs into routine calculations.

**When?**

Before diving into modelling or controller design—return here whenever you need to recall a formula.

**How?**

Review each subsection, run the linked Python snippets, and practice deriving the same quantities for your own example systems.

**Example**

Linearise pendulum dynamics using the Jacobian, then compare  $e^{At}$  from the matrix exponential with numerical integration to verify the approximation.

### 1.5.1 Single-Variable Calculus

We model systems with differential equations such as

$$\dot{x}(t) = f(x(t), u(t), t), \quad (1.1)$$

where  $x$  denotes the state and  $u$  the control input. Linearizing about an operating point  $(x_e, u_e)$  uses the first-order Taylor series

$$f(x, u) \approx f(x_e, u_e) + \left. \frac{\partial f}{\partial x} \right|_{(x_e, u_e)} (x - x_e) + \left. \frac{\partial f}{\partial u} \right|_{(x_e, u_e)} (u - u_e). \quad (1.2)$$

#### What?

A Taylor series approximates a smooth function by a polynomial whose coefficients are the derivatives of the function at an expansion point.

#### Why?

Polynomials are easy to manipulate, so replacing a nonlinear model with its Taylor approximation lets us reason about dynamics using linear algebra.

#### How?

Compute derivatives at  $(x_e, u_e)$ , plug them into the polynomial, and truncate after the desired order (first order for linearization).

#### When?

Whenever we analyse behaviour near an equilibrium or operating point—controllers rely on these local linear models.

To see the mechanics concretely, consider the scalar drag model  $f(x, u) = -cx^2 + u$  around  $(x_e, u_e) = (0, 0)$ . The derivative  $\frac{df}{dx} = -2cx$  evaluates to zero at the equilibrium, so the linear approximation reduces to  $\Delta\dot{x} \approx 1 \cdot \Delta u$ . This worked example clarifies why the Jacobian entries are simply slopes of  $f$  along each variable. In the multi-variable setting, the partial-derivative notation  $\frac{\partial f}{\partial x}$  denotes a row vector of partial derivatives with respect to every component of  $x$ ; the scripts in `scripts/dynamics.py` (e.g., `mass_spring_damper.linearize()`) automate this Jacobian evaluation. Run

```
from scripts.dynamics import mass_spring_damper
plant = mass_spring_damper()
print(plant.linearize_about_equilibrium())
```

to print both  $f(x_e, u_e)$  and the associated Jacobians, linking the pen-and-paper expansion to executable code.

### 1.5.2 Jacobian Matrices

The *Jacobian*  $J_f(x)$  of a vector-valued function  $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$  collects all first-order partial derivatives:

$$J_f(x) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix}.$$

**What?**

A compact way to measure how a multivariable function changes with respect to each input direction.

**Why?**

The Jacobian is the bridge between nonlinear systems and linear tools; it tells us how small perturbations map through the dynamics.

**How?**

Differentiate each output component with respect to each input variable; evaluate at the operating point.

**When?**

Use Jacobians to linearize nonlinear dynamics, propagate uncertainties (e.g., EKF), and compute sensitivities in optimisation.

Numerical example: linearize the pendulum dynamics  $f(\theta, \dot{\theta}) = [\dot{\theta}, -\frac{g}{\ell} \sin \theta]^\top$  near the upright position  $(\theta, \dot{\theta}) = (0, 0)$ . The Jacobian evaluates to

$$J_f(0, 0) = \begin{bmatrix} 0 & 1 \\ -\frac{g}{\ell} & 0 \end{bmatrix},$$

immediately revealing the familiar small-angle linear model. You can reproduce this computation with SymPy:

```
import sympy as sp
theta, dtheta, g, ell = sp.symbols("theta_dtheta_g_ell")
f = sp.Matrix([dtheta, -(g/ell)*sp.sin(theta)])
J = f.jacobian([theta, dtheta]).subs({theta: 0})
print(sp.simplify(J))
```

A similar Taylor expansion extends to multi-variable functions, with gradients and Jacobians capturing sensitivity.

### 1.5.3 Ordinary Differential Equations

Linear time-invariant (LTI) systems take the canonical form

$$\dot{x}(t) = Ax(t) + Bu(t), \quad y(t) = Cx(t) + Du(t), \quad (1.3)$$

where  $A \in \mathbb{R}^{n \times n}$  governs autonomous dynamics. The matrix exponential provides the closed-form solution

$$x(t) = e^{A(t-t_0)}x(t_0) + \int_{t_0}^t e^{A(t-\tau)}Bu(\tau) d\tau. \quad (1.4)$$

We will discretize continuous models using zero-order holds (ZOH) in Chapter 2 and in `scripts/dynamics.py`. To build intuition for the matrix exponential, expand the scalar first-order system  $\dot{x} = ax$  where the closed-form solution  $x(t) = e^{a(t-t_0)}x(t_0)$  is obtained by integrating  $\dot{x}/x = a$ . The multi-state formula above is the vector generalisation where  $e^{A(t-t_0)}$  replaces the scalar  $e^{a(t-t_0)}$ . The helper `plant.step_response()` prints both continuous and discrete predictions so you can compare them numerically.

**What?**

The matrix exponential  $e^{At}$  advances the state of a linear system over time  $t$ .

**How?**

Either compute the infinite series  $\sum_{k=0}^{\infty} \frac{(At)^k}{k!}$  or use numerical routines (`scipy.linalg.expm`) which leverage eigen-decompositions or scaling/squaring.

**When?**

Use it to solve continuous linear systems exactly, to discretize models for digital control, and to express sensitivity to sampling time.

Concrete example: consider  $A = \begin{bmatrix} 0 & 1 \\ -4 & -0.4 \end{bmatrix}$  (mass-spring-damper with  $m = 1$ ,  $k = 4$ ,  $c = 0.4$ ).

With  $T_s = 0.01$  s,

$$A_d = e^{AT_s} \approx \begin{bmatrix} 0.9990 & 0.00995 \\ -0.0398 & 0.9910 \end{bmatrix},$$

showing how the continuous dynamics map to discrete time. Verify numerically:

```
import numpy as np
from scipy.linalg import expm

A = np.array([[0.0, 1.0], [-4.0, -0.4]])
Ts = 0.01
Ad = expm(A * Ts)
print(np.round(Ad, 4))
```

The result matches the automatically generated matrices from `mass_spring_damper().discretize(Ts)`, reinforcing the link between analytic formulas and the course scripts.

**Matrix Exponential Recipes via Eigenstructure**

When  $A$  is diagonalizable ( $A = V\Lambda V^{-1}$  with  $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_n)$ ), the exponential is

$$e^{At} = V e^{\Lambda t} V^{-1}, \quad e^{\Lambda t} = \text{diag}(e^{\lambda_1 t}, \dots, e^{\lambda_n t}).$$

**Why?**

This formula reuses the scalar exponential on each eigenvalue and wraps the result back into the original coordinates through  $V$ .

**How to apply?**

Compute eigenvalues/eigenvectors (e.g., `np.linalg.eig`), form  $V$ , and evaluate the diagonal exponential.

If  $A$  is not diagonalizable, use the Jordan decomposition  $A = VJV^{-1}$  with Jordan blocks

$$J = \begin{bmatrix} J_1 & & 0 \\ & \ddots & \\ 0 & & J_k \end{bmatrix}, \quad J_\ell = \begin{bmatrix} \lambda_\ell & 1 & & 0 \\ & \lambda_\ell & \ddots & \\ & & \ddots & 1 \\ 0 & & & \lambda_\ell \end{bmatrix}.$$

Each Jordan block exponentiates to

$$e^{J_\ell t} = e^{\lambda_\ell t} \begin{bmatrix} 1 & t & \frac{t^2}{2!} & \cdots \\ 0 & 1 & t & \ddots \\ \vdots & \ddots & \ddots & \frac{t^2}{2!} \\ 0 & \cdots & 0 & 1 \end{bmatrix},$$

which follows from the nilpotent part  $N = J_\ell - \lambda_\ell I$  satisfying  $N^m = 0$  for some  $m$ .

### What does this buy us?

The Jordan form handles repeated eigenvalues and gives a constructive way to write  $e^{At}$  using polynomials in  $t$  times the familiar  $e^{\lambda t}$  factor.

### When to use it?

Whenever  $A$  has repeated eigenvalues without a full set of eigenvectors (common in controllable canonical forms or companion matrices).

### How to compute?

Libraries such as SymPy expose `.jordan_cells()` and `.exp()`, but the block formula above is short enough to implement manually for  $2 \times 2$  or  $3 \times 3$  cases.

Numerical illustration: a Jordan block with repeated eigenvalue  $-2$ ,

$$A = \begin{bmatrix} -2 & 1 \\ 0 & -2 \end{bmatrix}$$

has exponential

$$e^{At} = e^{-2t} \begin{bmatrix} 1 & t \\ 0 & 1 \end{bmatrix}.$$

Check against Python:

```
import sympy as sp
t = sp.symbols('t', real=True)
A = sp.Matrix([[ -2, 1],
               [ 0, -2]])
print(sp.exp(A*t))
```

The output matches the hand-derived block formula, validating the process.

**Linking Jacobians and Matrix Exponentials.** Linearizing a nonlinear system  $\dot{x} = f(x, u)$  at an equilibrium  $(x_e, u_e)$  produces the Jacobian  $A = \left. \frac{\partial f}{\partial x} \right|_{x_e, u_e}$ . This  $A$  governs local dynamics, so  $e^{At}$  describes how small perturbations around the equilibrium evolve. In practice:

1. Compute the Jacobian matrix with respect to the state.
2. Use eigenvalue or Jordan-based formulas to evaluate  $e^{At}$  (analytically or numerically).
3. Interpret the result: diagonal entries describe independent modes, while off-diagonal polynomial terms reveal coupling introduced by repeated eigenvalues.

This chain—*Jacobian*  $\rightarrow$  *state matrix*  $A \rightarrow$  *matrix exponential*—is central to every chapter that studies stability, discretization, or observer/controller design.

## 1.5.4 Linear Algebra Refresher

Key operations:

- **Eigenvalues** of  $A$  solve  $\det(A - \lambda I) = 0$  and determine the natural modes  $e^{\lambda t}$ .

- **Controllability** and **observability** utilize the matrices

$$\mathcal{C} = [B \ AB \ A^2B \ \dots \ A^{n-1}B], \quad \mathcal{O} = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix}. \quad (1.5)$$

Full rank for  $\mathcal{C}$  (rank =  $n$ ) ensures arbitrary state reachability; full rank for  $\mathcal{O}$  ensures the state can be reconstructed from outputs.

- **Singular Value Decomposition (SVD)** helps assess system gains and robustness.

### 1.5.5 Complex Exponentials and Transforms

Frequency-domain analysis relies on Euler's identity  $e^{j\omega t} = \cos \omega t + j \sin \omega t$ . The Laplace transform converts a differential equation to the algebraic relation

$$(sI - A)X(s) = x(0) + BU(s), \quad Y(s) = CX(s) + DU(s). \quad (1.6)$$

Discretization leads to the  $z$ -transform, which maps  $x_{k+1} = A_d x_k + B_d u_k$  to  $(zI - A_d)X(z) = x_0 + B_d U(z)$ .

## 1.6 Learning Roadmap

- **Module 1** formalizes modeling pipelines and demonstrates linearization and discretization in Python.
- **Module 2** analyzes time-domain response metrics—rise time, settling time, overshoot—with PID control demonstrations.
- **Module 3** pivots to frequency-response tools such as Bode and Nyquist plots.
- **Module 4** designs classical controllers (PID, lead/lag) with tuned simulations.
- **Modules 5–6** introduce modern state-space control, LQR, Kalman filtering, and adaptive methods.
- **Module 7** addresses digital implementation details, linking theory to practical deployment.
- **Module 8** explores data-driven modeling, PDE adaptive control, and nonlinear stability foundations.
- **Module 9** introduces Active Disturbance Rejection Control (ADRC) with observers and disturbance estimation inspired by Han [1] and Gao [2].
- **Suggested reading:** Åström and Hägglund [3] and Åström and Hägglund [4] for PID design intuition, Ljung [5] for the bridge between classical, data-driven, and state-space views, and Krstić et al. [6] for adaptive-control insights in both ODE and PDE settings.

### Key Takeaways



- Strong fundamentals in calculus, linear algebra, and complex analysis accelerate mastery of control concepts.
- Maintain a reproducible workflow: derive equations in  $\text{\LaTeX}$ , validate with Python, and include generated figures in the notes.

## 1.7 Keyword Review

### Keywords

- **Control loop:** the feedback structure that continually compares measured output with a reference.
- **State:** the smallest set of variables that fully describes the system at an instant.
- **Linearization:** using first-order Taylor series to approximate nonlinear dynamics near an equilibrium.
- **Matrix exponential:** the operator  $e^{At}$  that evolves linear state-space models forward in time.
- **Reproducible workflow:** the pairing of  $\text{\LaTeX}$  derivations with version-controlled Python experiments.



## Chapter 2

# Modeling Dynamic Systems

### Learning Objectives

- Build continuous-time models from first principles using Newton–Euler and energy-based methods.
- Linearize nonlinear dynamics about equilibria and express them in state-space and transfer-function forms.
- Discretize continuous-time systems, verify the results numerically, and understand modeling assumptions.

## 2.1 Concept Map for Beginners

### Concept Map

1. Start from the physical picture: identify forces, energy storage, and actuation so that each equation has a tangible meaning.
2. Translate physics into first-order state equations, emphasising the small handful of calculus and linear algebra identities that make the conversion systematic.
3. Close the loop with simulation: every derived matrix feeds directly into a Python helper to confirm intuition with numbers and plots.

## 2.2 Concept–Math–Engineering Loop

## 2.3 Mathematical Background and Modeling Assumptions

### What?

A roadmap for turning physical intuition into differential equations and state-space models.

### Why?

Clear assumptions and notation prevent algebraic mistakes and make it easier to share models with teammates.

| Concept                        | Mathematics                                   | Engineering Practice                                                                                                                      |
|--------------------------------|-----------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Modeling from first principles | Newton–Euler balance, Lagrange energy methods | Sketch free-body diagrams and verify with <code>mass_spring_damper</code> or the inverted pendulum notebook.                              |
| State-space representation     | Variable substitution, matrix assembly        | Use Python to compute $A$ , $B$ , $C$ , $D$ and double-check against manual derivations.                                                  |
| Discretization and validation  | Matrix exponential, zero-order hold integrals | Call <code>system.discretize(Ts)</code> and <code>simulate_discrete_system</code> to ensure discrete models match continuous predictions. |

Table 2.1: Core ideas in Module 2 and how they iterate between equations and code.

### When?

Before deriving dynamics for any new system—confirm variables, frames, and disturbance representations.

### How?

Choose meaningful coordinates, write balance equations, cast them into first-order form, and linearise around equilibria.

### Example

Define displacement and velocity for a mass–spring–damper, assume linear damping, and obtain  $\dot{x} = f(x, u)$  ready for controller design.

Mechanical, electrical, and thermal systems admit a common structure

$$\dot{x} = f(x, u, d, t), \quad y = h(x, u, t),$$

where  $x \in \mathbb{R}^n$  collects generalized coordinates and their derivatives,  $u$  is the control input, and  $d$  represents disturbances. Throughout the module we will:

1. choose  $x$  using minimal coordinates (e.g., position, velocity);
2. derive governing equations via Newton–Euler or Lagrange methods;
3. cast the dynamics into first-order state-space form;
4. linearize about equilibria and relate the result to transfer functions;
5. discretize the continuous model to interface with digital controllers.

The workflow mirrors the scalar Kalman derivation supplied in `kalman_deduction.tex`: begin with first principles, manipulate equations line-by-line, summarize the final form, and map each algebraic operation to executable Python.

## 2.4 Mass–Spring–Damper Derivation

### What?

A canonical second-order system used to illustrate the entire modelling pipeline.

### Why?

Its simplicity reveals each modelling step without unnecessary algebraic clutter, serving as a reusable template.

### When?

Whenever you need a baseline example to test new controllers or estimation ideas.

### How?

Apply Newton’s second law, convert to first-order state form, and validate with Python helpers.

### Example

Derive  $A = \begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix}$ , then call `mass_spring_damper().linearize_about_equilibrium()` to confirm the matrices match theory.

#### 2.4.1 Step 1: Coordinate and Force Definition

Let  $x$  be the displacement from equilibrium. Applying Newton’s second law to the mass  $m$  with spring constant  $k$  and viscous damping  $c$  gives

$$m\ddot{x} = -kx - c\dot{x} + u.$$

#### 2.4.2 Step 2: First-Order State Representation

Introduce the state vector  $x_1 = x$ ,  $x_2 = \dot{x}$ . The system becomes

$$\begin{aligned} \dot{x}_1 &= x_2, \\ \dot{x}_2 &= -\frac{k}{m}x_1 - \frac{c}{m}x_2 + \frac{1}{m}u. \end{aligned}$$

Stacking the equations yields

$$\dot{x} = \begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix} x + \begin{bmatrix} 0 \\ 1/m \end{bmatrix} u, \quad y = \begin{bmatrix} 1 & 0 \end{bmatrix} x.$$

#### 2.4.3 Step 3: Energy Cross-Check

The kinetic and potential energies,

$$T = \frac{1}{2}m\dot{x}^2, \quad V = \frac{1}{2}kx^2,$$

produce the same dynamics via Lagrange’s equation. This is a useful consistency check when multiple derivation paths exist.

| Python Code (snippet)                             | Mathematical Expression                                                                                           | Explanation                                                                     |
|---------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| <code>system = mass_spring_damper(m, c, k)</code> | $m\ddot{x} + c\dot{x} + kx = u$                                                                                   | Constructs the physical model parameters used in subsequent calculations.       |
| <code>system.A, system.B, system.C</code>         | $A = \begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix}, B = \begin{bmatrix} 0 \\ 1/m \end{bmatrix}, C = [1 \ 0]$ | Exposes the state-space matrices derived above.                                 |
| <code>system.transfer_function()</code>           | $G(s) = C(sI - A)^{-1}B + D$                                                                                      | Confirms equivalence between state-space and transfer-function representations. |

Table 2.2: Mapping the MSD derivation to `scripts/dynamics.py`.

#### 2.4.4 Final Formulas and Code Mapping

#### 2.4.5 Numerical Validation

Run the open-loop script:

```
from scripts.dynamics import mass_spring_damper
from scripts.simulation import simulate_discrete_system

system = mass_spring_damper(m=1.0, c=0.4, k=4.0)
Ts = 0.005
plant_d = system.discretize(Ts)
step = lambda k: 0.01 if k >= 1 else 0.0

result = simulate_discrete_system(
    plant_d.A, plant_d.B, plant_d.C,
    controller_step=lambda r, y, t: 0.0,
    ts=Ts, steps=2000, reference=step,
)
print(result.performance_metrics())
```

Confirm that the response agrees with the analytic solution

$$x(t) = e^{At}x(0) + \int_0^t e^{A(t-\tau)}Bu(\tau) d\tau.$$

### 2.5 Inverted Pendulum Linearization

#### What?

A nonlinear benchmark that demonstrates Jacobian-based linearisation for an underactuated system.

#### Why?

Many real-world plants resemble the cart-pendulum; understanding its linearisation builds intuition for balancing problems.

#### When?

Before designing controllers such as LQR or swing-up strategies that assume a linear model around the upright equilibrium.

**How?**

Derive the nonlinear equations, compute partial derivatives at the equilibrium, and assemble  $A$  and  $B$  matrices.

**Example**

Use the supplied SymPy notebook to generate  $A$  and  $B$ , then verify controllability in Python.

**2.5.1 Step 1: Nonlinear Dynamics**

For the cart–pendulum system,

$$(M + m)\ddot{x} + m\ell\ddot{\theta}\cos\theta - m\ell\dot{\theta}^2\sin\theta = u,$$

$$\ddot{\theta} + \frac{g}{\ell}\sin\theta = -\frac{\ddot{x}}{\ell}\cos\theta.$$

**2.5.2 Step 2: Linearization About Upright Equilibrium**

Set  $(x, \theta, \dot{x}, \dot{\theta}) = (0, 0, 0, 0)$  and  $u = 0$ . Compute Jacobians:

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{mg}{M} & 0 & 0 \\ 0 & \frac{(M+m)g}{M\ell} & 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 0 \\ 1/M \\ 1/(M\ell) \end{bmatrix}.$$

These are obtained by differentiating each component  $f_i$  of  $\dot{x} = f(x, u)$  with respect to the state and input, mirroring the step-by-step gradients shown in `kalman_deduction.tex`.

**2.5.3 Step 3: Code Mapping**

| Python                                            | Formula                   | Comment                                         |
|---------------------------------------------------|---------------------------|-------------------------------------------------|
| <code>notebooks/01_linearization_of_dyn.py</code> | $\partial f / \partial u$ | SymPy script reproduces each Jacobian entry.    |
| <code>system.linearize()</code>                   | $A, B$ above              | Verifies the linearized model programmatically. |

Table 2.3: Inverted pendulum linearization traceability.

**2.6 Transfer-Function Connection****What?**

The algebra that links state-space models to transfer functions and vice versa.

**Why?**

Switching representations lets you use whichever analysis tool—time or frequency domain—is most convenient.

**When?**

After deriving a state-space model but before plotting Bode or Nyquist diagrams that rely on transfer functions.

**How?**

Take Laplace transforms, solve for  $X(s)$  and  $Y(s)$ , and reduce the expression to  $G(s) = C(sI - A)^{-1}B + D$ .

**Example**

Show that the mass-spring-damper produces  $G(s) = \frac{1/m}{s^2 + (c/m)s + k/m}$ , matching the transfer function computed by `system.transfer_function()`.

**2.6.1 Step-by-Step Derivation**

Starting from the state-space model  $(A, B, C, D)$ , take Laplace transforms (zero initial state for simplicity):

$$\begin{aligned} sX(s) &= AX(s) + BU(s), \\ Y(s) &= CX(s) + DU(s). \end{aligned}$$

Solve for  $X(s) = (sI - A)^{-1}BU(s)$  and substitute into the output:

$$G(s) = \frac{Y(s)}{U(s)} = C(sI - A)^{-1}B + D.$$

For the MSD parameters  $(m, c, k)$ ,

$$G(s) = \frac{1/m}{s^2 + (c/m)s + k/m}.$$

The poles at  $s = -\frac{c}{2m} \pm j\sqrt{k/m - c^2/(4m^2)}$  reveal the system's natural frequency and damping ratio, providing checks against time-domain intuition.

**2.6.2 Summary**

- State-space and transfer-function forms are two sides of the same coin for LTI systems.
- The inverse  $(sI - A)^{-1}$  encodes modal behaviour; each pole of  $G(s)$  corresponds to an eigenvalue of  $A$ .
- Use `system.transfer_function()` to confirm algebraic manipulations numerically.

**2.7 Zero-Order Hold Discretization****What?**

Converting continuous dynamics to discrete time under piecewise-constant inputs.

**Why?**

Digital controllers sample signals; accurate discretisation preserves stability and performance predictions.

**When?**

After modelling a continuous plant and before implementing discrete control laws.

**How?**

Evaluate  $e^{AT_s}$  and the integral for  $B_d$ , often using the block-matrix exponential trick.



**Example**

Compute discrete matrices for the mass–spring–damper at  $T_s = 5$  ms and compare with `system.discretize(Ts)` to confirm numerical agreement.

**2.7.1 Derivation**

Assume  $u(t)$  is constant over  $[kT_s, (k+1)T_s)$ :

$$\begin{aligned} x((k+1)T_s) &= e^{AT_s}x(kT_s) + \int_0^{T_s} e^{A\tau} B \, d\tau \, u[k] \\ &\equiv A_d x[k] + B_d u[k]. \end{aligned}$$

To compute  $A_d$  and  $B_d$ , form the block matrix

$$M = \begin{bmatrix} A & B \\ 0 & 0 \end{bmatrix}, \quad e^{MT_s} = \begin{bmatrix} A_d & B_d \\ 0 & I \end{bmatrix}.$$

**2.7.2 Numerical Example**

With  $m = 1$ ,  $c = 0.4$ ,  $k = 4$ ,  $T_s = 5$  ms,

$$A_d = \begin{bmatrix} 0.9990 & 0.00499 \\ -0.0199 & 0.9960 \end{bmatrix}, \quad B_d = \begin{bmatrix} 1.25 \times 10^{-5} \\ 0.00499 \end{bmatrix}.$$

Verify via `SystemMatrix.discretize(Ts)` in `scripts/dynamics.py` or with SciPy’s `expm`.

**2.7.3 Code Mapping**

| Python                                          | Formula                                                      | Explanation                                                                     |
|-------------------------------------------------|--------------------------------------------------------------|---------------------------------------------------------------------------------|
| <code>plant_d = system.discretize(Ts)</code>    | $A_d = e^{AT_s}$ , $B_d = \int_0^{T_s} e^{A\tau} B \, d\tau$ | Helper computes the matrix exponential and integral via the block-matrix trick. |
| <code>simulate_discrete_system(x_k, u_k)</code> | $x_{k+1} = A_d x_k + B_d u_k$                                | Simulator uses the discrete model directly for validation.                      |

Table 2.4: Discretization formulas versus code.

**2.8 Model Validation, Identification, and Refinement****What?**

Techniques for tuning model parameters and checking fidelity against data.

**Why?**

Even elegant derivations drift from reality—validation ensures controllers act on trustworthy models.

**When?**

After initial modelling and whenever real measurements reveal mismatches.

**How?**

Fit parameters via residual minimisation, analyse innovations, and augment models when systematic errors remain.

**Example**

Use the identification notebook to estimate damping  $c$  from step-response data and observe the improvement in simulation accuracy.

- **Parameter identification:** fit unknown constants (e.g.,  $c$ ,  $k$ ) by minimizing the squared innovation between measured and simulated outputs. See `notebooks/02_identification.ipynb`.
- **Unmodelled dynamics:** append additional poles/zeros or disturbance states when residuals exhibit systematic structure.
- **Model reduction:** apply balanced truncation to high-order finite-element models to retain dominant modes without overburdening real-time controllers.

## 2.9 Hands-On Practice

**What?**

Guided exercises to reinforce modelling concepts with executable experiments.

**Why?**

Practice cements theory and reveals practical issues like sensitivity to parameter changes.

**When?**

Immediately after reading the chapter—hands-on repetition shortens the feedback loop between learning and application.

**How?**

Run the provided tutorial scripts, perturb parameters, and inspect the resulting figures.

**Example**

Execute `python -m scripts.tutorials modeling --output-dir figures` and compare the generated step plot with analytic predictions.

1. Generate comparison plots with `python -m scripts.tutorials modeling --output-dir figures`. Inspect `figures/modeling_step.png` to verify discretization accuracy.
2. Perturb  $m$ ,  $c$ , or  $k$  and re-run the tutorial to see how pole locations and transient metrics move.
3. Evaluate the innovation sequence from `simulate_discrete_system` when parameter identification is active; use it to tune modelling assumptions before designing controllers.

Appendix B lists the exact `make` targets and plotting commands if you need a refresher while working through these exercises.

**Key Takeaways**

- Start from physics, linearize carefully, and always connect algebra to executable code.
- Transfer functions and state-space models provide complementary insights—use both.
- Discretization bridges continuous models with digital controllers; validate numerically before deployment.

## 2.10 Keyword Review

### Keywords

- **State-space model:** the pair  $(A, B, C, D)$  describing  $\dot{x} = Ax + Bu$ ,  $y = Cx + Du$ .
- **Linearization:** first-order Taylor expansion that turns nonlinear dynamics into an approximate linear model.
- **Transfer function:**  $G(s) = C(sI - A)^{-1}B + D$  mapping inputs to outputs in the Laplace domain.
- **Zero-order hold:** the assumption of piecewise-constant inputs used when discretizing continuous systems.
- **Model validation:** comparing simulated responses and innovations against data to judge fidelity.



## Chapter 3

# Time-Domain Analysis

### Learning Objectives

- Judge stability of linear systems using eigenvalue analysis, Lyapunov methods, and characteristic polynomials.
- Relate transient specifications (rise time, settling time, overshoot) to system poles and damping ratios.
- Follow complete numerical examples that compute performance metrics and validate them with Python simulations.

## 3.1 Concept Map for Beginners

### Concept Map

1. Begin with qualitative questions—does the system settle, oscillate, or diverge?—and tie them directly to eigenvalues and characteristic polynomials.
2. Translate performance requirements into pole locations using a small library of formulas for damping ratio and natural frequency.
3. Verify every analytical claim with the provided simulations, reinforcing algebra by observing the same effect in plots and logged metrics.

## 3.2 Concept–Math–Engineering Loop

## 3.3 Stability by Eigenvalues and Lyapunov Functions

### What?

Analytical tools for determining whether system trajectories decay, persist, or diverge.

### Why?

Stability is the first safety check—controllers are useless if the closed loop explodes.

### When?

Before tuning performance; confirm the open- or closed-loop poles sit in the stable region.

| Concept                   | Mathematics                                                | Engineering Practice                                                                                             |
|---------------------------|------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| Stability assessment      | Eigenvalues, Lyapunov equation, Routh–Hurwitz              | Run eigenvalue checks and Lyapunov solves in Python to classify dynamics quickly.                                |
| Transient specifications  | Second-order step response formulas, damping ratio $\zeta$ | Adjust $\zeta$ and $\omega_n$ in <code>tutorial_time_domain</code> to see predicted overshoot and settling time. |
| Controller interpretation | Root locus algebra, PID difference equations               | Compare analytic root locus steps with the scripted plots and PID experiment logs.                               |

Table 3.1: How Module 3 balances theory, computation, and hands-on validation.

**How?**

Inspect eigenvalues of  $A$  or solve Lyapunov equations to obtain energy-like certificates.

**Example**

Compute  $P$  from  $A^\top P + PA = -Q$  for a second-order system and verify numerically that  $V(x) = x^\top Px$  decreases along trajectories.

**3.3.1 Step-by-Step Derivation**

The homogeneous system  $\dot{x} = Ax$  has solution  $x(t) = e^{At}x(0)$ . Eigenvalues of  $A$  govern behaviour:

- $\text{Re}(\lambda_i) < 0$  for all  $i \Rightarrow$  asymptotically stable.
- Eigenvalues on the imaginary axis with no repetitions  $\Rightarrow$  marginally stable.
- Any  $\text{Re}(\lambda_i) > 0$  or repeated imaginary-axis pole  $\Rightarrow$  unstable.

For a constructive certificate, choose  $Q = Q^\top \succ 0$  and solve the Lyapunov equation

$$A^\top P + PA = -Q.$$

If  $P \succ 0$ , the quadratic function  $V(x) = x^\top Px$  decreases along trajectories, proving asymptotic stability.

**3.3.2 Code Mapping**

| Python                                                       | Formula              | Explanation                                                                                     |
|--------------------------------------------------------------|----------------------|-------------------------------------------------------------------------------------------------|
| <code>np.linalg.eigvals(A)</code>                            | $\lambda_i(A)$       | Computes eigenvalues used to categorise stability.                                              |
| <code>scipy.linalg.solve_continuous_lyapunov(A.T, -Q)</code> | $A^\top P + PA = -Q$ | Returns $P$ ; check positive definiteness with <code>np.all(np.linalg.eigvals(P)&gt;0)</code> . |

Table 3.2: Eigenvalue and Lyapunov stability checks.

## 3.4 Characteristic Polynomials and Routh–Hurwitz

### What?

A tabular test for assessing polynomial stability without explicit root finding.

### Why?

High-order characteristic equations arise frequently; Routh–Hurwitz provides a fast manual or symbolic check.

### When?

When designing classical controllers that alter pole locations via parameter tuning.

### How?

Construct the Routh table coefficients and ensure the first column remains positive.

### Example

Apply the procedure to a cubic closed-loop polynomial and confirm the conditions using `sympy.routh_array()`.

#### 3.4.1 Derivation Workflow

For a unity-feedback loop with open-loop transfer  $L(s)$ , the closed-loop characteristic polynomial is  $D(s) + N(s)$ . Routh–Hurwitz builds a tabular test without root finding. Example for  $s^3 + a_1s^2 + a_2s + a_3$ :

$$\begin{array}{c|cc} s^3 & 1 & a_2 \\ s^2 & a_1 & a_3 \\ s^1 & \frac{a_1a_2 - a_3}{a_1} & 0 \\ s^0 & a_3 & \end{array}$$

Every first-column entry must be positive. The template generalises to higher orders; replicate the calculation symbolically with SymPy to cement the algebra.

#### 3.4.2 Code Mapping

| Python                                | Formula               | Explanation                                                    |
|---------------------------------------|-----------------------|----------------------------------------------------------------|
| <code>sympy.routh_array(poly)</code>  | Routh table entries   | Automates the eliminations; inspect intermediate determinants. |
| <code>numpy.poly(stable_roots)</code> | $D(s)$ reconstruction | Useful for cross-checking with explicit pole placement.        |

Table 3.3: Routh–Hurwitz computations in Python.

## 3.5 Second-Order Response Metrics

### What?

Closed-form relations between damping ratio, natural frequency, and time-domain specifications.

**Why?**

These approximations convert qualitative requirements (overshoot, settling time) into numeric design targets.

**When?**

Early in controller tuning to estimate suitable pole locations or PID gains.

**How?**

Use the standard second-order step response equations and solve for  $\zeta$  and  $\omega_n$  that meet specs.

**Example**

Given  $M_p \leq 10\%$  and  $t_s \leq 1.5$  s, solve for  $\zeta \approx 0.59$ ,  $\omega_n \geq 4.5$  rad/s, then simulate to validate.

**3.5.1 Derivation**

Starting from

$$G(s) = \frac{\omega_n^2}{s^2 + 2\zeta\omega_n s + \omega_n^2},$$

use inverse Laplace transforms to obtain

$$y(t) = 1 - \frac{1}{\sqrt{1-\zeta^2}} e^{-\zeta\omega_n t} \sin(\omega_d t + \phi),$$

with  $\omega_d = \omega_n \sqrt{1-\zeta^2}$ . Differentiating this step response yields classic metrics:

$$\begin{aligned} t_r &\approx \frac{1.8}{\omega_n} & 0.4 \leq \zeta \leq 0.8, \\ t_p &= \frac{\pi}{\omega_d}, \\ M_p &= e^{-\pi\zeta/\sqrt{1-\zeta^2}}, \\ t_s &\approx \frac{4}{\zeta\omega_n} & (2\% \text{ criterion}). \end{aligned}$$

**3.5.2 Worked Specification Example**

Design for  $t_s \leq 1.5$  s and  $M_p \leq 10\%$ :

1. Solve  $0.10 = e^{-\pi\zeta/\sqrt{1-\zeta^2}} \Rightarrow \zeta \approx 0.59$ .
2. Impose  $t_s \approx 4/(\zeta\omega_n) \leq 1.5$  to get  $\omega_n \geq 4.5$  rad/s.
3. Target dominant poles at  $-\zeta\omega_n \pm j\omega_d$  with  $\omega_d = \omega_n \sqrt{1-\zeta^2} \approx 3.7$  rad/s.

**3.5.3 Code Mapping****3.6 Root-Locus Construction****What?**

A graphical method showing how closed-loop poles move as a scalar gain varies.



| Python                                             | Formula                   | Explanation                                              |
|----------------------------------------------------|---------------------------|----------------------------------------------------------|
| <code>control.step_info(G)</code>                  | $t_r, t_p, M_p, t_s$      | Computes metrics from the transfer function.             |
| tutorial_time_domain<br>in<br>scripts/tutorials.py | $y(t)$ vs. design targets | Visualises the effect of changing $\zeta$ , $\omega_n$ . |

Table 3.4: Second-order performance checks.

**Why?**

It reveals stability margins and helps choose gains without exhaustive simulation.

**When?**

During classical controller design, especially when adjusting proportional gain or placing lead/lag networks.

**How?**

Plot the locus using pole/zero locations, angle criteria, and asymptote formulas.

**Example**

Sketch the locus for  $K/(s(s^2 + 0.4s + 4))$ , then confirm with `control.root_locus()` to see the poles leave the real axis.

**3.6.1 Manual Derivation**

For  $L(s) = K \frac{N(s)}{D(s)}$ , root-locus points satisfy  $1 + L(s) = 0$ . The steepest steps mirror the derivation style in `kalman_deduction.tex`:

1. Identify pole/zero structure of  $L(s)$ .
2. Apply angle criterion  $\angle L(s) = (2k + 1)\pi$  to determine locus on the real axis.
3. Compute asymptotes:  $\sigma = (\sum \text{poles} - \sum \text{zeros})/(n - m)$ ,  $\theta_k = \frac{(2k+1)\pi}{n-m}$ .
4. Evaluate  $\frac{dK}{ds} = 0$  to locate breakaway points.

**3.6.2 Example**

For  $L(s) = K/(s(s^2 + 0.4s + 4))$ , differentiating  $K(s) = -s(s^2 + 0.4s + 4)$  gives  $3s^2 + 0.8s + 4 = 0$ , whose complex roots show that the locus leaves the real axis immediately.

**3.6.3 Code Mapping****3.7 PID Controllers: Time-Domain Interpretation****What?**

The time-domain view of proportional-integral-derivative control and its discrete implementation.

**Why?**

Understanding how each term shapes error evolution informs tuning and anti-windup strategies.

| Python                                                      | Formula              | Explanation                                      |
|-------------------------------------------------------------|----------------------|--------------------------------------------------|
| <code>control.root_locus(L)</code>                          | Locus plot           | Visualises steps 1–4 graphically.                |
| <code>tutorial_time_domain<br/>generate_root_locus()</code> | Annotated asymptotes | Recreates the analytical derivation numerically. |

Table 3.5: Root-locus derivation and verification.

**When?**

Whenever you deploy PID loops or need a baseline controller before moving to advanced methods.

**How?**

Derive the difference equation, implement it with saturation guards, and tune gains using response metrics.

**Example**

Use `PIDController.print_incremental_coefficients()` to inspect discrete coefficients and simulate the mass–spring–damper case to see their effect.

**3.7.1 Incremental Implementation**

The discrete incremental PID law is

$$u[k] = u[k-1] + K_p(e[k] - e[k-1]) + K_i T_s e[k] + \frac{K_d}{T_s}(e[k] - 2e[k-1] + e[k-2]).$$

Derivative filtering and anti-windup strategies (clamping, back-calculation) ensure robustness to quantization and saturation.

**3.7.2 Simulation Case Study**

```
from scripts.experiments import EXPERIMENTS

pid_result = EXPERIMENTS["pid"].runner()
print(pid_result.performance_metrics())
```

Typical metrics include a steady-state error of  $\approx 6 \times 10^{-3}$  and overshoot near 75% for the aggressive tuning. Compare these with second-order predictions to tune  $K_p, K_i, K_d$  systematically.

**3.7.3 Code Mapping**

| Python                            | Formula             | Explanation                                                    |
|-----------------------------------|---------------------|----------------------------------------------------------------|
| <code>PIDController.step()</code> | Incremental PID law | Implements the difference equation and anti-windup hooks.      |
| <code>tutorial_time_domain</code> | $y(t), u(t)$ plots  | Shows how gain changes alter $t_s, M_p$ , and actuator effort. |

Table 3.6: PID controller equations versus code.

## 3.8 Disturbance and Noise Rejection

### What?

Analysis of how feedback loops attenuate load disturbances and amplify measurement noise.

### Why?

Controllers must balance robustness against disturbances with immunity to sensor imperfections.

### When?

After satisfying nominal performance, evaluate disturbance/noise behaviour before hardware deployment.

### How?

Examine sensitivity functions  $S$  and complementary sensitivity  $T$ , augment simulations with injected disturbances.

### Example

Modify `tutorial_time_domain` to add a step disturbance at  $t = 2$  s and compare the resulting  $y(t)$  across PID tunings.

#### 3.8.1 Derivation

For unity feedback,

$$Y(s) = \frac{P(s)}{1 + P(s)C(s)}D(s) + \frac{P(s)C(s)}{1 + P(s)C(s)}R(s).$$

Low-frequency disturbances are attenuated when  $|PC| \gg 1$ ; sensor noise amplification grows with high-frequency loop gain, highlighting the need for derivative filtering.

#### 3.8.2 Case Study

Modify `tutorial_time_domain` to inject a step disturbance at  $t = 2$  s and Gaussian sensor noise with variance  $10^{-4}$ . Compare the resulting plots under aggressive and conservative PID tunings to visualise the sensitivity functions  $S = 1/(1 + PC)$  and  $T = PC/(1 + PC)$ .

## 3.9 Workflow Summary and Practice

### What?

A checklist for applying time-domain techniques end-to-end.

### Why?

Summaries help ensure you validate assumptions, tune gains responsibly, and document results reproducibly.

### When?

After completing the chapter and whenever you start a new design iteration.

### How?

Follow the enumerated steps: map specs to poles, verify stability, cross-check with simulation, and log results.

**Example**

Run `python -m scripts.tutorials time --output-dir figures`, annotate rise time/overshoot from the plots, and compare with second-order predictions.

1. Translate specifications into pole targets using the second-order formulas and root-locus insights.
2. Verify algebraic stability tests (Lyapunov, Routh–Hurwitz) with Python to avoid arithmetic mistakes.
3. Validate against simulations using `python -m scripts.tutorials time --output-dir figures`; inspect `figures/time_pid_responses.png` and `figures/time_pid_controls.png`.
4. Log innovations or residuals in experiments to tune disturbance/noise models before moving to frequency-domain design.

Refer back to Appendix B for the reproducible plotting commands and `make` targets that automate these checks.

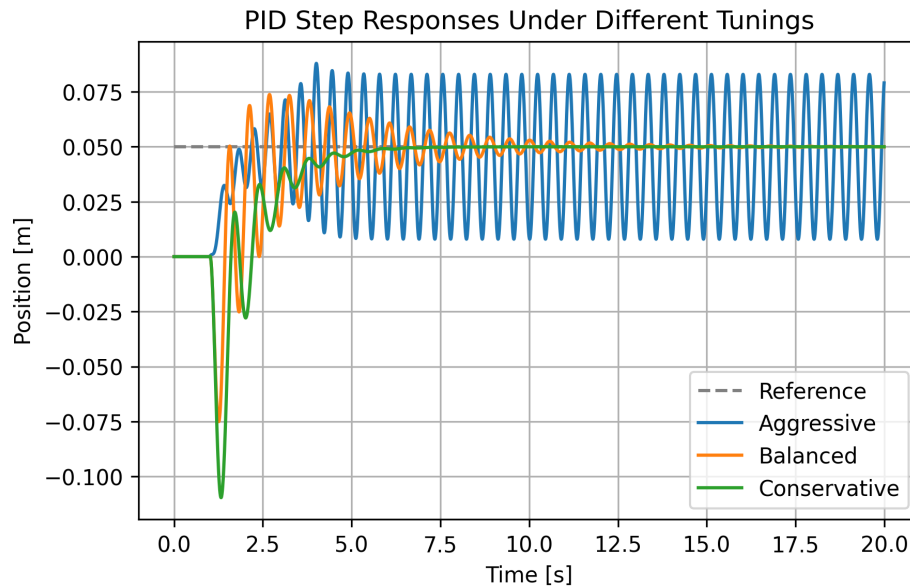


Figure 3.1: PID step responses for multiple tunings generated via `python -m scripts.tutorials time`.

**Key Takeaways**

- Stability analysis combines algebraic tests (Routh–Hurwitz) with geometric intuition (root locus, eigenvalues).
- Performance formulas for second-order systems provide quick design rules which must be cross-checked against simulations.
- Python-backed experiments translate equations into numerical intuition, highlighting effects of nonidealities.

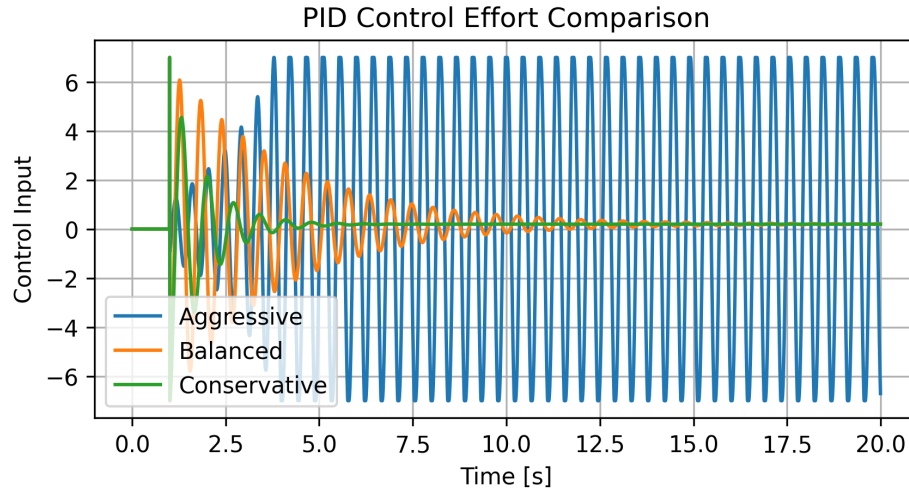


Figure 3.2: Control effort comparison across PID tunings.

### 3.10 Module Review

- **Stability:** eigenvalue locations, Lyapunov equation  $A^T P + PA = -Q$ .
- **Routh–Hurwitz:** all first-column entries positive  $\Leftrightarrow$  closed-loop stability.
- **Second-order metrics:**  $t_s \approx 4/(\zeta\omega_n)$ ,  $M_p = e^{-\pi\zeta/\sqrt{1-\zeta^2}}$ .
- **Root locus:** asymptote centroid  $\sigma$  and angles  $\theta_k$  describe pole migration.
- **PID implementation:** incremental difference equation with anti-windup safeguards.
- **Disturbance rejection:**  $S = 1/(1 + PC)$ ,  $T = PC/(1 + PC)$  quantify load disturbance versus noise amplification.

### 3.11 Keyword Review

#### Keywords

- **Asymptotic stability:** all eigenvalues with negative real parts (continuous time) or magnitude less than one (discrete time).
- **Lyapunov function:** energy-like scalar used to certify stability; decreases along trajectories.
- **Characteristic polynomial:** determinant  $\det(sI - A)$  (or closed-loop denominator) whose roots are system poles.
- **Damping ratio  $\zeta$ :** dimensionless measure controlling overshoot and oscillation in second-order systems.
- **Root locus:** plot showing closed-loop pole trajectories as a gain varies.
- **Sensitivity  $S$ :** transfer function mapping disturbances to output, highlighting robustness limits.



## Chapter 4

# Frequency-Domain Foundations

### Learning Objectives

- Interpret the sinusoidal steady-state response of linear time-invariant (LTI) systems and connect it to Bode, Nyquist, and Nichols plots.
- Compute gain/phase margins, crossover frequencies, and sensitivity functions from analytic expressions and sampled data.
- Carry out complete worked examples that start from a transfer function, construct frequency plots, and verify results numerically using Python.

## 4.1 Concept Map for Beginners

### Concept Map

1. View frequency response as a question about how the system reacts to sine waves—once that story is clear, complex algebra becomes less intimidating.
2. Build Bode and Nyquist plots step-by-step, always relating slope changes or encirclements to the underlying poles and zeros.
3. Use the tutorial scripts to overlay analytic sketches with high-resolution plots so you can see how each formula shapes a tangible curve.

## 4.2 Concept–Math–Engineering Loop

## 4.3 Sinusoidal Steady-State Response

### What?

Describes how linear systems respond to sinusoidal inputs by evaluating  $G(j\omega)$ .

### Why?

Frequency response directly connects to Bode and Nyquist plots used for robustness analysis.

### When?

Before constructing frequency-domain plots or computing gain/phase margins.

| Concept                 | Mathematics                                         | Engineering Practice                                                                               |
|-------------------------|-----------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Sinusoidal steady state | Evaluate $G(j\omega)$ and interpret magnitude/phase | Execute <code>system.frequency_response</code> to compare analytic and numerical values.           |
| Loop-shaping plots      | Bode asymptotes, Nyquist encirclement rules         | Generate tutorial figures to overlay manual sketches with exact curves.                            |
| Robustness metrics      | Gain/phase margins, sensitivity functions           | Use <code>control.margin</code> and sensitivity utilities to tie algebra to margin specifications. |

Table 4.1: Frequency-domain learning workflow balancing concepts, formulas, and tooling.

**How?**

Substitute  $s = j\omega$ , compute magnitude and phase, and compare with simulation.

**Example**

Evaluate  $|G(j\omega)|$  for the mass–spring–damper plant using `system.frequency_response(omega)` and overlay analytic formulas.

**4.3.1 Derivation**

Excite an LTI system with  $u(t) = U \sin(\omega t)$ . The steady-state output is

$$y(t) = |G(j\omega)|U \sin(\omega t + \angle G(j\omega)),$$

where  $G(s) = C(sI - A)^{-1}B + D$  is the transfer function. Substituting  $s = j\omega$  and simplifying the complex arithmetic yields magnitude and phase expressions, exactly as demonstrated in the mass–spring–damper case study below.

**4.3.2 Code Mapping**

| Python                                               | Formula                              | Explanation                                              |
|------------------------------------------------------|--------------------------------------|----------------------------------------------------------|
| <code>system.frequency_response(omega)</code>        | $G(j\omega)$                         | Uses linear algebra to evaluate $(j\omega I - A)^{-1}$ . |
| <code>np.abs(G)</code> ,<br><code>np.angle(G)</code> | $ G(j\omega) $ , $\angle G(j\omega)$ | Direct comparison with analytic expressions.             |

Table 4.2: Complex-frequency evaluation pipeline.

**4.4 Bode Plot Construction****What?**

A graphical summary of magnitude and phase versus frequency.



**Why?**

Bode plots reveal bandwidth, resonances, and the impact of controller poles/zeros at a glance.

**When?**

During loop-shaping to ensure sufficient gain/phase margins and noise attenuation.

**How?**

Factor  $G(s)$ , sketch asymptotes, and verify with computational tools.

**Example**

Plot the mass–spring–damper response via `python -m scripts.tutorials frequency` and annotate break frequencies predicted analytically.

**4.4.1 Analytic Template**

Factor the transfer function

$$G(s) = K s^{n_z} \prod_{i=1}^{N_z} \left(1 + \frac{s}{z_i}\right) \prod_{i=1}^{N_p} \left(1 + \frac{s}{p_i}\right)^{-1}$$

and sketch the magnitude by superimposing straight-line asymptotes: each pole contributes  $-20$  dB/decade beyond its break frequency; zeros add  $+20$  dB/decade. The phase is the sum of arctangent contributions.

**4.4.2 Worked Example: Mass–Spring–Damper**

For  $m = 1$ ,  $c = 0.4$ ,  $k = 4$ ,

$$|G(j\omega)| = \frac{1}{\sqrt{(4 - \omega^2)^2 + (0.4\omega)^2}},$$

$$\angle G(j\omega) = -\tan^{-1}\left(\frac{0.4\omega}{4 - \omega^2}\right).$$

Verify the algebra using the short script in the original notes and confirm numerically with the helper below.

**4.4.3 Python Reproduction**

```
import numpy as np
import matplotlib.pyplot as plt
from scripts.dynamics import mass_spring_damper

system = mass_spring_damper(m=1.0, c=0.4, k=4.0)
omega = np.logspace(-1, 2, 600)
Gjw = system.frequency_response(omega)

plt.figure()
plt.semilogx(omega, 20*np.log10(np.abs(Gjw)))
plt.xlabel("Frequency [rad/s]"); plt.ylabel("Magnitude [dB]")
plt.grid(True, which="both")

plt.figure()
plt.semilogx(omega, np.degrees(np.angle(Gjw)))
```

```
plt.xlabel("Frequency [rad/s]"); plt.ylabel("Phase [deg]")
plt.grid(True, which="both")
plt.show()
```

## 4.5 Nyquist Criterion and Stability Margins

### What?

A contour-based test that links open-loop frequency response to closed-loop stability.

### Why?

It provides gain and phase margins critical for robust controller tuning.

### When?

After choosing a controller candidate but before finalising gains.

### How?

Plot  $L(j\omega)$ , count encirclements of  $-1$ , and read margins from intersections.

### Example

Use `control.nyquist(L)` on the PID-regulated mass–spring–damper to visualise margin improvements.

#### 4.5.1 Derivation

The Nyquist contour maps the right-half plane through  $L(s) = C(s)P(s)$ . Encirclements of  $-1$  encode closed-loop stability via  $N = Z - P$ . Gain and phase margins follow from the intersections of  $L(j\omega)$  with the unit circle and negative real axis:

$$\begin{aligned} \text{GM} &= \frac{1}{|L(j\omega_\pi)|}, \quad \angle L(j\omega_\pi) = -\pi, \\ \text{PM} &= \pi + \angle L(j\omega_c), \quad |L(j\omega_c)| = 1. \end{aligned}$$

#### 4.5.2 Case Study: PID-Regulated MSD

With  $K_p = 40$ ,  $K_i = 80$ ,  $K_d = 2$ ,

$$L(s) = \frac{K_d s^2 + K_p s + K_i}{s(s^2 + 0.4s + 4)}.$$

Evaluate  $L(j\omega)$  at sample frequencies to observe how the locus approaches  $-1$ . The Nyquist plot generated by:

```
from scripts.tutorials import frequency
frequency.generate_nyquist_overlay(output_dir="figures")
```

overlays plant and compensated loops, emphasising margin improvements.

| Python                          | Formula                 | Explanation                                                   |
|---------------------------------|-------------------------|---------------------------------------------------------------|
| <code>control.nyquist(L)</code> | $L(j\omega)$ trajectory | Visualises encirclements and margins.                         |
| <code>control.margin(L)</code>  | GM, PM                  | Numerically extracts $\omega_c$ , $\omega_\pi$ , and margins. |

Table 4.3: Nyquist analysis tooling.

### 4.5.3 Code Mapping

## 4.6 Sensitivity, Nichols, and Loop-Shaping

### What?

Tools for quantifying disturbance rejection ( $S$ ) and noise amplification ( $T$ ) along with Nichols charts.

### Why?

Sensitivity functions clarify robustness trade-offs beyond simple margin numbers.

### When?

After verifying stability and margins to fine-tune robustness and performance.

### How?

Compute  $S$  and  $T$  across frequency, inspect Nichols overlays, and adjust controller shape accordingly.

### Example

Call `frequency.compute_sensitivity(L)` to plot  $|S|$  and  $|T|$ , confirming low-frequency disturbance rejection for the tuned PID.

### 4.6.1 Key Relations

$$S(j\omega) = \frac{1}{1 + L(j\omega)}, \quad T(j\omega) = \frac{L(j\omega)}{1 + L(j\omega)}.$$

Nichols charts plot loop gain (dB) versus phase and overlay contours of constant  $|S|$  and  $|T|$  to visualise robustness trade-offs.

### 4.6.2 Numerical Example

For the PID design above:

$$|S(j0.1)| \approx 0.05, \quad |S(j1)| \approx 0.22, \quad |S(j10)| \approx 0.99,$$

highlighting excellent low-frequency disturbance rejection and modest high-frequency noise amplification.

| Python                                        | Formula                  | Explanation                                                                |
|-----------------------------------------------|--------------------------|----------------------------------------------------------------------------|
| <code>frequency.compute_sensitivity(L)</code> | $S(j\omega), T(j\omega)$ | Utility in <code>scripts/tutorials</code> evaluates sensitivity functions. |
| <code>control.nichols(L)</code>               | Nichols chart            | Displays contours for loop-shaping decisions.                              |

Table 4.4: Sensitivity analysis utilities.

### 4.6.3 Code Mapping

## 4.7 Practice Workflow

### What?

A repeatable sequence for applying frequency-domain analysis.

### Why?

Structured steps reduce mistakes when moving from algebra to plots and back.

### When?

Whenever you iterate on loop-shaping or document design decisions.

### How?

Derive analytic expressions, generate plots with tutorial scripts, measure margins, and record sensitivity insights.

### Example

Generate `figures/frequency_nyquist_overlay.png` and highlight the distance to  $-1$  at crossover to support your gain choice.

1. Derive  $|G(j\omega)|$  and  $\angle G(j\omega)$  analytically, then verify with `system.frequency_response`.
2. Construct Bode plots and annotate break frequencies; compare with control-library results.
3. Produce Nyquist and Nichols diagrams via `python -m scripts.tutorials frequency --output-dir figures`; inspect `figures/frequency_bode_overlay.png`, `frequency_nyquist_overlay.png`, and `frequency_root_locus.png`.
4. Extract gain and phase margins, ensuring  $\text{GM} \geq 6$  dB and  $\text{PM} \geq 45^\circ$  for baseline robustness.
5. Evaluate  $S$  and  $T$  to balance disturbance rejection against noise amplification before finalising controller parameters.

Appendix B documents the exact figure-generation workflow so you can regenerate the overlays without hunting for command syntax.

### Key Takeaways

- Frequency-domain analysis provides complementary intuition to time-domain metrics, especially for robustness and disturbance rejection.

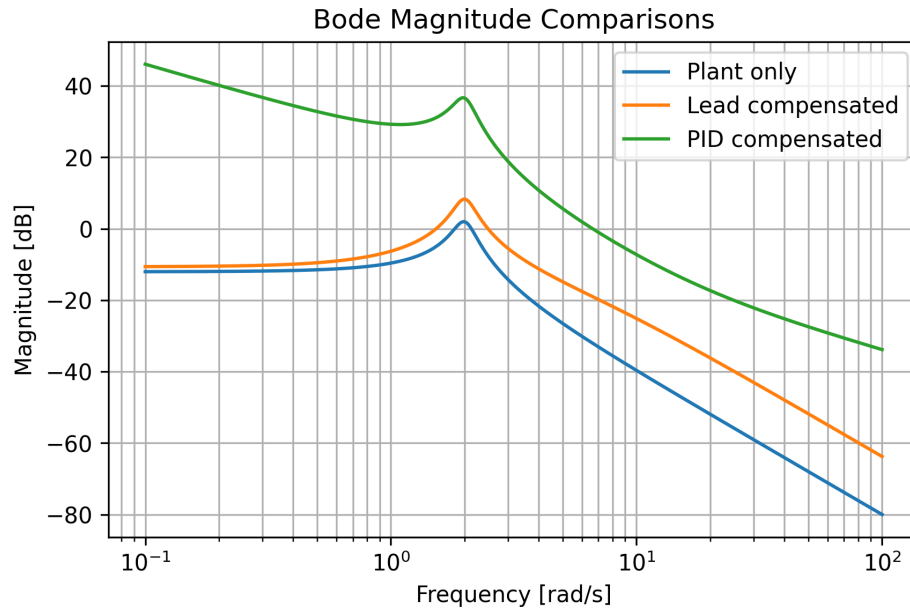


Figure 4.1: Bode magnitude comparison generated by `python -m scripts.tutorials frequency`.

- Worked examples demonstrate how to transition from formulas to concrete plots and numerical values; repeat the workflow on your own systems.
- Use Python scripts to automate frequency sweeps, ensuring consistency between analytical derivations and visual intuition.

## 4.8 Module Review

- **Core relations:**  $G(j\omega) = C(j\omega I - A)^{-1}B + D$ , gain margin and phase margin from Nyquist, sensitivity functions  $S = 1/(1 + L)$ ,  $T = L/(1 + L)$ .
- **Design lens:** correlate Bode slope changes with pole/zero placements, ensure Nyquist trajectory maintains recommended margins (45–60° phase per Åström and Hägglund [4]).
- **Validation checklist:** compare analytic sketches with tutorial overlays, cross-reference time-domain metrics with frequency margins, and inspect root-locus movement under gain variation.
- **Practice:** run `python -m scripts.tutorials frequency` and annotate key crossover frequencies and distances to  $-1$  on the Nyquist plot.
- **Further reading:** Åström and Hägglund [4] Ch. 4–5 and Ljung [5] Ch. 7 for loop-shaping methodology.

## 4.9 Keyword Review

Keywords

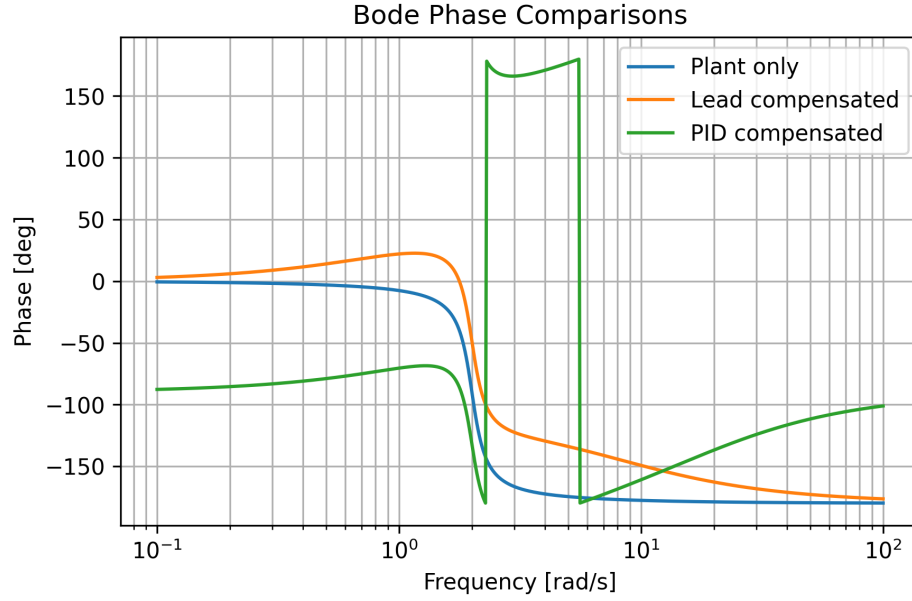


Figure 4.2: Bode phase overlay generated by the tutorial script.

- **Bode plot:** log-frequency magnitude and phase visualization built from superposed pole/zero contributions.
- **Gain margin (GM):** factor by which loop gain can increase before instability ( $GM = 1/|L(j\omega_\pi)|$ ).
- **Phase margin (PM):** additional phase lag allowed at gain crossover ( $PM = \pi + \angle L(j\omega_c)$ ).
- **Nyquist criterion:** encirclement rule linking open-loop frequency response to closed-loop pole counts.
- **Nichols chart:** gain-versus-phase plot with sensitivity contours for loop shaping.
- **Sensitivity  $S$ :** magnitude indicates disturbance rejection;  $|S| < 1$  implies attenuation at that frequency.

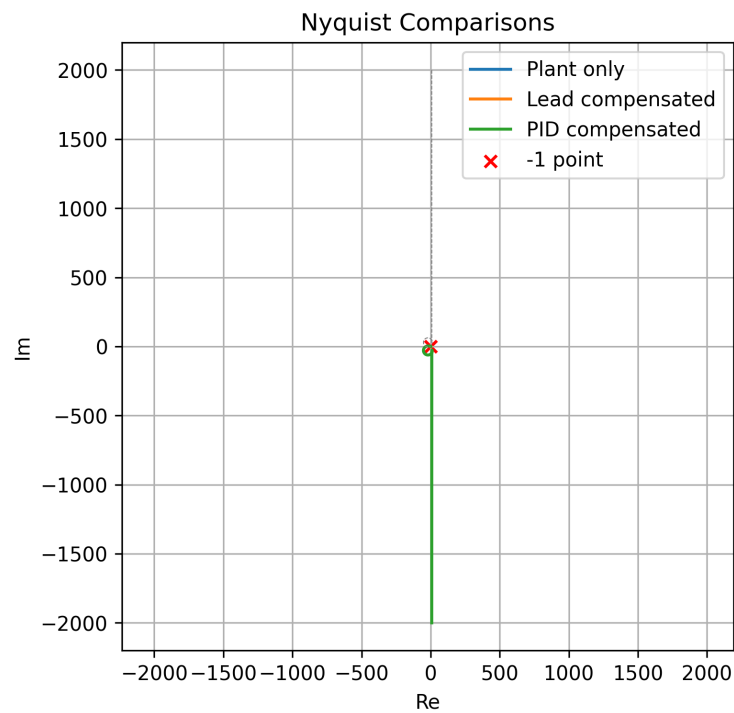


Figure 4.3: Nyquist overlays illustrating loop-shape changes across controllers.

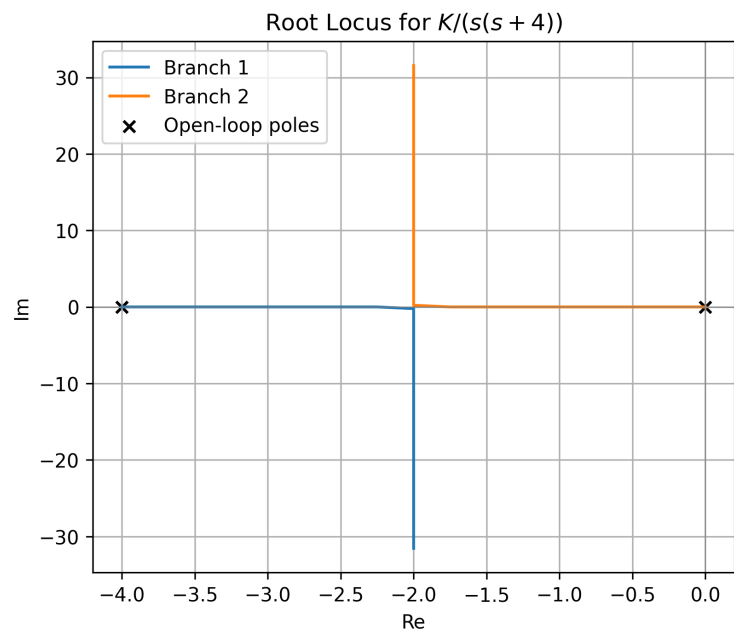


Figure 4.4: Root locus of  $K/(s(s + 4))$  illustrating pole migration as  $K$  varies.





## Chapter 5

# Classical Controller Design

### Learning Objectives

- Derive continuous and discrete PID controllers, understand tuning guidelines, and identify practical limitations.
- Shape loop dynamics with lead, lag, and lead–lag compensators to achieve robustness and steady-state requirements.
- Follow detailed case studies that connect analytic calculations to Python-based validation.

## 5.1 Concept Map for Beginners

### Concept Map

1. Interpret PID gains as levers on rise time, steady-state error, and noise; translate every algebraic step into a physical effect.
2. Use structured tuning templates (Ziegler–Nichols, direct synthesis) to move from specifications to controller parameters.
3. Validate designs through simulation plots and logged metrics, reinforcing the idea that analytic formulas are hypotheses verified in code.

## 5.2 Concept–Math–Engineering Loop

## 5.3 PID Controller Derivation

### What?

Derives continuous and discrete PID laws, relating Laplace-domain formulas to implementable difference equations.

### Why?

PID remains the industry workhorse; understanding its derivation clarifies how each term shapes dynamics.

| Concept          | Mathematics                                           | Engineering Practice                                                                                                          |
|------------------|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| PID synthesis    | Laplace-domain derivation, Tustin discretization      | Inspect coefficients via <code>PIDController.print_incremental_coefficients</code> before deploying.                          |
| Empirical tuning | Ultimate gain/period analysis, reaction-curve algebra | Run <code>demo.py pid</code> while applying Ziegler–Nichols or Cohen–Coon tunes to match observed oscillations.               |
| Lead/lag design  | Pole-zero placement, angle condition                  | Overlay Bode/Nyquist plots using <code>scripts.tutorials.classical</code> to see phase boosts and low-frequency gain changes. |

Table 5.1: Linking classical controller concepts to the math and Python validation workflow.

**When?**

Before tuning gains or porting a controller to embedded code.

**How?**

Express  $C(s)$ , apply discretisation (e.g., Tustin), and reorganise into an incremental update with anti-windup hooks.

**Example**

Use the derived coefficients in `PIDController.step()` and observe how derivative filtering changes overshoot in simulation.

**5.3.1 Continuous-Time Formula**

The classic PID law

$$C(s) = K_p + \frac{K_i}{s} + K_d s = K_p \left( 1 + \frac{1}{T_i s} + T_d s \right)$$

combines proportional, integral, and derivative actions. Proportional gain speeds up response, integral action removes steady-state error, and derivative action adds phase lead but amplifies sensor noise without filtering.

**5.3.2 Discrete Realization**

Applying the Tustin substitution  $s \rightarrow \frac{2}{T_s} \frac{1-z^{-1}}{1+z^{-1}}$  yields

$$C(z) = K_p \left( 1 + \frac{T_s}{2T_i} \frac{1+z^{-1}}{1-z^{-1}} + \frac{2T_d}{T_s} \frac{1-z^{-1}}{1+z^{-1}} \right).$$

Expanding and grouping powers of  $z^{-1}$  provides the incremental implementation:

$$\begin{aligned} u[k] = & u[k-1] + K_p \left( 1 + \frac{T_s}{2T_i} + \frac{2T_d}{T_s} \right) e[k] \\ & - K_p \left( 1 - \frac{T_s}{2T_i} + \frac{4T_d}{T_s} \right) e[k-1] \\ & + K_p \left( \frac{2T_d}{T_s} \right) e[k-2]. \end{aligned}$$

### 5.3.3 Code Mapping

| Python                                            | Formula             | Explanation                                                                    |
|---------------------------------------------------|---------------------|--------------------------------------------------------------------------------|
| <code>PIDController.step()</code>                 | Incremental PID law | Implements the discrete update with saturation and derivative filtering hooks. |
| <code>pid.print_incremental_coefficients()</code> | Coefficient readout | Verifies algebraic manipulation term-by-term.                                  |

Table 5.2: PID difference equation traceability.

## 5.4 Empirical Tuning Templates

### What?

Rule-of-thumb PID parameter sets (Ziegler–Nichols, Cohen–Coon, direct synthesis).

### Why?

They provide reliable starting points when system identification or optimisation is unavailable.

### When?

During commissioning or when you need a quick baseline before fine-tuning.

### How?

Measure ultimate gain/period or reaction-curve parameters, plug into the formulas, and validate with simulation.

### Example

Determine  $K_u$  and  $T_u$  from a sustained oscillation test, then apply Table 5.3 to initialise  $K_p$ ,  $T_i$ ,  $T_d$ .

#### 5.4.1 Ziegler–Nichols

Increase  $K_p$  with  $K_i = K_d = 0$  until sustained oscillations occur; record the ultimate gain  $K_u$  and period  $T_u$ . Recommended starting gains appear in Table 5.3.

| Controller | $K_p$     | $(T_i, T_d)$               |
|------------|-----------|----------------------------|
| P          | $0.5K_u$  | –                          |
| PI         | $0.45K_u$ | $T_i = T_u/1.2$            |
| PID        | $0.6K_u$  | $T_i = T_u/2, T_d = T_u/8$ |

Table 5.3: Ziegler–Nichols initial settings.

#### 5.4.2 Cohen–Coon

For processes with appreciable delay, identify the reaction-curve parameters (dead time  $L$ , rise time  $R$ , process gain  $K$ ) and substitute them into the Cohen–Coon formulas (referenced in Åström and

Hägglund [3]). These provide better balance between overshoot and steady-state accuracy when lag dominates.

### 5.4.3 Direct Synthesis

Choose a desired closed-loop model  $T(s)$  and back out the controller:

$$C(s) = \frac{T(s)}{1 - T(s)} \frac{1}{P(s)}.$$

Factor the result into PID plus filters. This method mirrors optimal-control intuition and prepares learners for state-space design.

## 5.5 Lead/Lag Compensation

### What?

Methods for shaping phase and steady-state gain using lead, lag, or lead-lag networks.

### Why?

These compensators meet robustness and accuracy requirements without abandoning classical design tools.

### When?

After baseline PID design when extra phase margin or steady-state accuracy is needed.

### How?

Place zeros/poles analytically, compute required gain, and verify margins via Bode/Nyquist plots.

### Example

Choose  $\alpha$  for a desired phase boost, set  $z$  and  $p$ , then evaluate `control.margin(K * compensator * P)` to confirm targets.

### 5.5.1 Lead Design Steps

1. Target crossover  $\omega_c$  and evaluate  $P(j\omega_c)$ .
2. Determine additional phase  $\phi_{\text{req}}$  to meet the desired phase margin.
3. Choose  $\alpha = \frac{1 - \sin \phi_{\text{req}}}{1 + \sin \phi_{\text{req}}}$ ; place zero at  $z = \omega_c / \sqrt{\alpha}$  and pole at  $p = \alpha z$ .
4. Select gain  $K$  so that  $|KC_{\text{lead}}(j\omega_c)P(j\omega_c)| = 1$ .

The lead transfer function  $C_{\text{lead}}(s) = K \frac{s+z}{s+p}$  boosts phase by  $\phi_{\text{max}} = \sin^{-1} \left( \frac{1-\alpha}{1+\alpha} \right)$  near  $\sqrt{zp}$ .

### 5.5.2 Lag Design Steps

Lag networks increase low-frequency gain via  $C_{\text{lag}}(s) = K \frac{s+z}{s+p}$  with  $z/p \approx 10$  and  $p$  placed below crossover. They improve steady-state accuracy with modest phase penalty.

| Python                                           | Formula              | Explanation                                                          |
|--------------------------------------------------|----------------------|----------------------------------------------------------------------|
| <code>control.tf([1/z, 1], [1/p, 1])</code>      | $C_{\text{lead}}(s)$ | Constructs lead or lag compensators directly from zero/pole choices. |
| <code>control.margin(K * compensator * P)</code> | GM, PM               | Validates the analytic gain/phase calculations.                      |

Table 5.4: Lead/lag synthesis via Python.

### 5.5.3 Code Mapping

## 5.6 Case Study: Second-Order Position Control

### What?

A complete worked example applying lead compensation to a second-order plant.

### Why?

Demonstrates how analytic formulas translate into numerical gains and measurable performance metrics.

### When?

Use as a template when tackling similar servo problems or presenting design rationale.

### How?

Analyse the uncompensated plant, select crossover, compute compensator parameters, and verify via simulation.

### Example

Implement the provided Python script to compute GM/PM and rise time for the compensated  $P(s) = 1/(s(s+4))$  plant.

For  $P(s) = \frac{1}{s(s+4)}$  and requirements  $t_r < 0.6$  s,  $M_p < 15\%$ ,  $\text{PM} > 50^\circ$ :

1. Uncompensated analysis shows insufficient phase margin.
2. Choose  $\omega_c = 3$  rad/s; evaluate  $|P(j3)| \approx 0.11$  and phase  $\approx -129^\circ$ .
3. Required phase boost  $\approx 35^\circ \Rightarrow \alpha = 0.18$ ,  $z \approx 1.3$ ,  $p \approx 7.2$ ,  $K \approx 6.5$ .
4. Append lag (e.g., zero at 0.03, pole at 0.003) if steady-state error remains large.

### 5.6.1 Simulation Verification

```
import control, numpy as np

P = control.tf([1], [1, 4, 0])
lead = control.tf([1/1.3, 1], [1/7.2, 1])
K = 6.5
L = K * lead * P
GM, PM, _, _ = control.margin(L)
print(f"GM={20*np.log10(GM):.2f} dB, PM={PM:.1f} deg")
T = control.feedback(L)
t, y = control.step_response(T)
rt = t[np.searchsorted(y, 0.9)] - t[np.searchsorted(y, 0.1)]
print(f"Rise time: {rt:.3f} s")
```

Compare simulated metrics with design predictions and adjust  $K$ ,  $z$ ,  $p$  accordingly.

## 5.7 Practice Workflow

### What?

A step-by-step procedure for classical controller design with simulation validation.

### Why?

Ensures you iterate systematically and capture both analytical and numerical evidence.

### When?

After completing derivations and before presenting calibration results.

### How?

Derive equations, apply tuning templates, shape loops, run tutorial scripts, and document outcomes.

### Example

Execute `python -m scripts.tutorials classical --output-dir figures` and annotate the resulting step plot with achieved specs.

1. Derive continuous and discrete PID equations; verify coefficients with `PIDController.print_incremental_coe`
2. Apply Ziegler–Nichols or Cohen–Coon to obtain starting gains, then refine using second-order metrics from Chapter 3.
3. Design lead/lag compensators using the analytic steps; check gain/phase margins with `control.margin`.
4. Run `python -m scripts.tutorials classical --output-dir figures` to generate comparison plots such as Figure 5.1.
5. Overlay uncompensated and compensated responses to visualise how loop shaping meets specifications.

See Appendix B for the canonical command snippets that regenerate these plots and manage build artifacts.

## 5.8 Classical and Modern Connections

### What?

Conceptual bridge between frequency-domain compensators and state-space feedback/observers.

### Why?

Recognising these parallels eases the transition to modern control and unifies the course content.

### When?

After mastering classical tools and before tackling LQR or Kalman filter chapters.

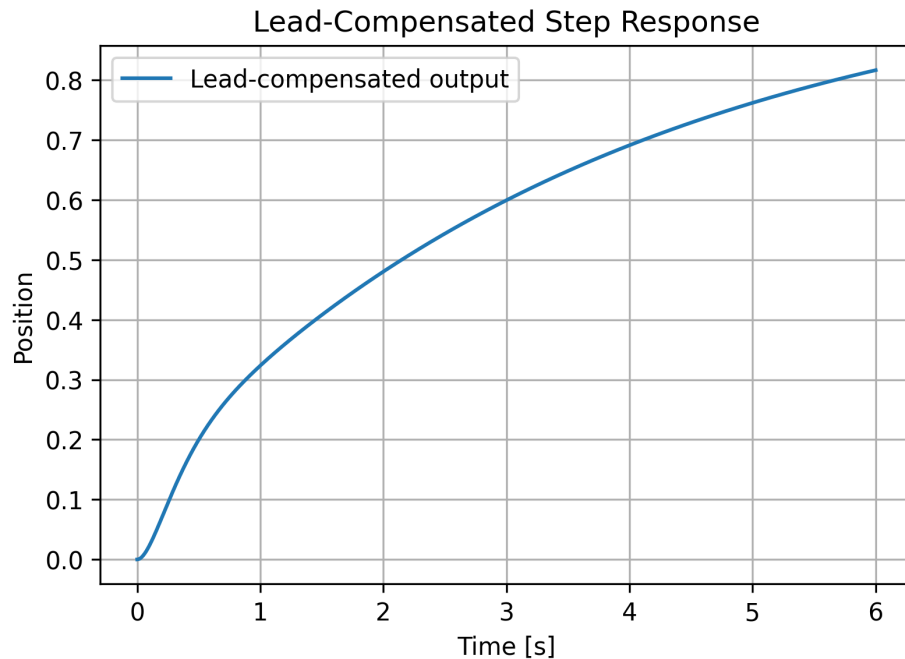


Figure 5.1: Lead-compensated step response created by `python -m scripts.tutorials.classical`.

### How?

Interpret integral action as state augmentation, lead compensation as pole placement, and lag as bias correction.

### Example

Rewrite a PI controller as state feedback on  $[x; \int e]$  and compare gains with a full state-space design.

Classical compensators can be interpreted via state augmentation:

- Integral action corresponds to augmenting the state with  $x_I = \int e(t) dt$  and applying state feedback  $u = -Kx - K_I x_I$ .
- Lead compensation approximates state-feedback placement of dominant poles combined with prefilters.
- Lag compensation resembles observer-based bias correction by shaping low-frequency gain.

This perspective eases the transition to Chapter 6, where optimal control and observers generalize these concepts. Ljung [5] emphasizes bridging frequency-domain intuition with state-space design for learners moving toward multi-variable systems.

### Key Takeaways

- PID controllers provide a versatile baseline; tune them thoughtfully and implement anti-windup.

- Lead and lag networks enable robust shaping of loop gain and phase, ensuring compliance with stability margins.
- Analytic design must always be cross-checked with simulation: use Bode, Nyquist, and step responses to validate specifications.

## 5.9 Module Review

- **PID law:**  $u[k]$  incremental form with Tustin-discretised coefficients.
- **Empirical tuning:** Ziegler–Nichols and Cohen–Coon rules provide quick starting points.
- **Lead/lag:** phase boost  $\phi_{\max}$ , zero/pole placement, gain selection to hit  $\omega_c$ .
- **Validation:** frequency-domain margins and time-domain step response must both meet specifications.
- **Practice:** leverage `python -m scripts.tutorials classical` to iterate on analytic design choices.

## 5.10 Keyword Review

### Keywords

- **PID controller:** combination of proportional, integral, and derivative actions shaping loop dynamics.
- **Ultimate gain/period** ( $K_u$ ,  $T_u$ ): oscillation parameters used for Ziegler–Nichols tuning.
- **Lead compensator:** zero-pole pair that adds phase around crossover to increase stability margins.
- **Lag compensator:** zero-pole pair that increases low-frequency gain to reduce steady-state error.
- **Incremental implementation:** discrete-time difference equation computing  $u[k]$  from recent errors to limit numerical drift.



## Chapter 6

# State-Space Methods and Modern Control

### Learning Objectives

- Evaluate controllability and observability rigorously and understand their physical meaning.
- Design state-feedback controllers via pole placement and Linear Quadratic Regulation (LQR), including servo prefilters.
- Integrate state estimation (Kalman filtering) to close the loop in the presence of measurement noise.

## 6.1 Concept Map for Beginners

### Concept Map

1. Treat the state as the story of the system between input and output; once that narrative is clear, matrices simply formalize it.
2. Diagnose controllability and observability before designing anything so you know which goals are achievable.
3. Pair optimal control (LQR) with optimal estimation (Kalman filter) to build the modern separation-principle controller seen in practice.

## 6.2 Concept–Math–Engineering Loop

The presentation mirrors the step-by-step development advocated by Ljung [5], blending geometric insight with computational recipes.

## 6.3 Why State-Space?

### What?

Explains the benefits of representing dynamics with internal state variables.

| Concept                       | Mathematics                                   | Engineering Practice                                                                                                                     |
|-------------------------------|-----------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| Keeping the state explicit    | Canonical realisations, matrix assembly       | Recreate $A, B, C, D$ from a transfer function with <code>control.ss</code> to confirm intuition.                                        |
| Deciding what is controllable | Rank of $\mathcal{C}, \mathcal{O}$            | Use <code>demo.py lqr --no-plot</code> to print ranks before attempting placement or LQR.                                                |
| Optimal control + estimation  | Discrete Riccati equations for LQR and Kalman | Run <code>EXPERIMENTS["lqr"]</code> and <code>EXPERIMENTS["kf_lqr"]</code> to inspect gains, covariance logs, and innovation statistics. |

Table 6.1: The Module 6 workflow linking conceptual reasoning, algebra, and executable experiments.

### Why?

State-space handles multivariable systems, initial conditions, and modern control designs more naturally than transfer functions alone.

### When?

Whenever systems have multiple inputs/outputs, need observer design, or involve optimal control.

### How?

Define the state, write  $\dot{x} = Ax + Bu$ ,  $y = Cx + Du$ , and relate it to familiar transfer-function descriptions.

### Example

Convert a mass–spring–damper transfer function to controllable canonical form using `control.ss`.

Transfer functions describe how inputs map to outputs by eliminating intermediate variables. State-space models keep those internal variables—the *state*—explicit:

$$\dot{x} = Ax + Bu, \quad (6.1)$$

$$y = Cx + Du. \quad (6.2)$$

For beginners, it helps to answer three practical questions:

- **What is the state?** A minimal set of variables that completely describe the system at any instant. For a mass–spring–damper,  $x = [\text{position}; \text{velocity}]$  is enough to predict future motion.
- **When should I use state-space?** Whenever you need to model multiple inputs/outputs, track initial conditions explicitly, or design modern controllers (state feedback, observers, optimal control). Classical transfer-function tools become cumbersome in these scenarios.
- **How does it relate to transfer functions?** If a system is linear and time invariant, you can move between the two representations. Eliminating the state from the equations produces the transfer function; factoring a transfer function back into first-order equations yields a state-space realisation.

### 6.3.1 Connecting to Transfer Functions

Consider a single-input single-output (SISO) transfer function

$$G(s) = \frac{b_m s^m + \cdots + b_1 s + b_0}{s^n + a_{n-1} s^{n-1} + \cdots + a_1 s + a_0},$$

normalised so the leading denominator coefficient is 1. One controllable canonical realisation is obtained by defining the state as the stacked derivatives of the output,

$$x = \begin{bmatrix} y \\ \dot{y} \\ \vdots \\ y^{(n-1)} \end{bmatrix},$$

producing the matrices

$$A = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ -a_0 & -a_1 & -a_2 & \cdots & -a_{n-1} \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix}, \quad (6.3)$$

$$C = [\beta_0 \ \beta_1 \ \cdots \ \beta_{n-1}], \quad D = d, \quad (6.4)$$

where the strictly proper case ( $m \leq n - 1$ ) sets  $d = 0$  and  $\beta_i = b_i$  (pad missing coefficients with zeros so the vector has length  $n$ ). If the numerator order equals the denominator order, factor out the direct term  $d = b_n$  and build the strictly proper remainder using the formula above. This construction guarantees that  $G(s) = C(sI - A)^{-1}B + D$ .

### 6.3.2 Worked Example: Mass–Spring–Damper

The plant  $G(s) = \frac{1/m}{s^2 + \frac{c}{m}s + \frac{k}{m}}$  yields

$$A = \begin{bmatrix} 0 & 1 \\ -k/m & -c/m \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 1/m \end{bmatrix}, \quad C = [1 \ 0], \quad D = 0,$$

matching the physics-based derivation in Chapter 2. The Python snippet

```
from control import tf, ss
G = tf([1], [1, 0.4, 4])      # m = 1, c = 0.4, k = 4
A, B, C, D = ss(G)
print(A)
print(B)
```

confirms the algebra numerically and reinforces the connection between the two viewpoints. Try modifying the transfer function (e.g., adding zeros) and watch how the canonical matrices change.

## 6.4 Controllability and Observability

### What?

Criteria indicating whether inputs can move every state and outputs can reconstruct them.

**Why?**

Feedback and observers fail if the system is uncontrollable or unobservable; these checks prevent wasted design effort.

**When?**

Immediately after forming  $(A, B, C, D)$  and before attempting pole placement or estimation.

**How?**

Compute the controllability matrix  $\mathcal{C}$  and observability matrix  $\mathcal{O}$  and inspect their ranks.

**Example**

Use `EXPERIMENTS["lqr"].runner()` to print the ranks for the discretised mass–spring–damper, confirming readiness for control design.

The controllability matrix

$$\mathcal{C} = [B \ AB \ A^2B \ \dots \ A^{n-1}B] \quad (6.5)$$

has full rank ( $n$ ) if the state can be driven to any point in finite time. Observability relies on

$$\mathcal{O} = \begin{bmatrix} C \\ CA \\ \vdots \\ CA^{n-1} \end{bmatrix}, \quad (6.6)$$

which must also have rank  $n$  to reconstruct the state from outputs. Physically, uncontrollable states correspond to modes unaffected by actuators; unobservable states correspond to hidden modes in the measurements.

**6.4.1 Numerical Check**

Use the Python experiment

```
python demo.py lqr --no-plot
```

which prints ranks of  $\mathcal{C}$  and  $\mathcal{O}$  for the discretized mass–spring–damper system, confirming controllability and observability.

**6.5 Pole Placement****What?**

A method for assigning closed-loop eigenvalues via state feedback.

**Why?**

It gives direct control over system dynamics when full-state feedback is available.

**When?**

After verifying controllability and before exploring optimal control alternatives like LQR.

**How?**

Use Ackermann's formula or numerical solvers to compute  $K$  such that  $A - BK$  has desired eigenvalues.

**Example**

Call `scipy.signal.place_poles` on a controllable canonical system to place poles at  $[-2, -3]$  and compare simulated responses.

Design a feedback law  $u = -Kx$  such that  $A - BK$  has desired eigenvalues  $\{\lambda_i\}$ . Ackermann's formula for single-input systems yields

$$K = e_n^T \mathcal{C}^{-1} \phi_d(A), \quad (6.7)$$

where  $e_n^T = [0 \ 0 \ \dots \ 1]$  and  $\phi_d(A)$  denotes the desired characteristic polynomial evaluated at  $A$ . Practical workflow:

1. Choose desired poles based on transient requirements (e.g., match damping ratio and natural frequency).
2. Use `scipy.signal.place_poles` or control library to compute  $K$  and verify eigenvalues numerically.
3. Simulate closed-loop response to ensure no actuator saturation or excessive control effort occurs.

## 6.6 Linear Quadratic Regulation

**What?**

An optimal control law balancing state deviations and control effort via the Riccati equation.

**Why?**

LQR offers systematic tuning knobs ( $Q$ ,  $R$ ) and guarantees closed-loop stability for controllable systems.

**When?**

After baseline pole placement when you want performance-effort trade-offs and robustness improvements.

**How?**

Solve the discrete or continuous algebraic Riccati equation, compute  $K$ , and optionally add a prefilter for tracking.

**Example**

Evaluate `EXPERIMENTS["lqr"].runner()` to see how different  $Q/R$  choices alter transient response and control usage.

LQR minimizes the infinite-horizon cost

$$J = \sum_{k=0}^{\infty} (x_k^T Q x_k + u_k^T R u_k) \quad (6.8)$$

subject to  $x_{k+1} = Ax_k + Bu_k$ . The solution involves:

1. Solving the discrete algebraic Riccati equation (DARE)

$$P = A^T P A - A^T P B (B^T P B + R)^{-1} B^T P A + Q. \quad (6.9)$$

2. Computing  $K = (B^T P B + R)^{-1} B^T P A$ .
3. Using  $u_k = -K x_k$  to close the loop; the closed-loop eigenvalues equal those of  $A - BK$  and lie inside the unit circle.

Select  $Q$  and  $R$  to penalize state deviations and control effort. Increasing  $Q$  entries tightens state regulation; increasing  $R$  reduces control energy at the expense of slower response. To show where the Riccati equation originates, start from the finite-horizon cost

$$J_N = x_N^T P_N x_N + \sum_{k=0}^{N-1} (x_k^T Q x_k + u_k^T R u_k)$$

and minimise it by substituting  $u_k = -K_k x_k$ . Completing the squares yields the backward recursion

$$\begin{aligned} P_k &= Q + A^T P_{k+1} A - A^T P_{k+1} B (R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A, \\ K_k &= (R + B^T P_{k+1} B)^{-1} B^T P_{k+1} A, \end{aligned}$$

with terminal condition  $P_N = P_N^T \succeq 0$ . As  $N \rightarrow \infty$ , the matrices converge to  $P$  and  $K$  that satisfy the steady-state DARE above. Add a short logging loop to the LQR experiment (or call the recursion interactively) to print each  $P_k$  so beginners can watch the sequence settle numerically before trusting the steady-state solver.

### 6.6.1 Servo Prefilter

To track constant references without steady-state error, compute the prefilter gain

$$N_{\text{bar}} = \frac{1}{C(I - A + BK)^{-1}B} \quad (6.10)$$

for single-output systems. The helper `discrete_nbar` automates this calculation. The control law becomes  $u_k = -K x_k + N_{\text{bar}} r_k$ .

## 6.7 Kalman Filtering in Depth

### What?

Optimal state estimation under linear-Gaussian assumptions.

### Why?

Accurate state estimates enable state-feedback controllers when not all states are measured and quantify uncertainty for diagnostics.

### When?

After specifying noise covariances and before closing the loop with LQR or other controllers.

### How?

Propagate predictions with  $A$ ,  $B$ , update with measurements via the Kalman gain, and tune using innovation statistics.

### Example

Run `EXPERIMENTS["kf_lqr"].runner(return_logs=True)` to collect innovations and verify tuning against the theoretical covariance.

The Kalman filter is a probabilistic extension of the Luenberger observer that accounts for process and measurement uncertainty. We begin with the stochastic model

$$x_{k+1} = Ax_k + Bu_k + w_k, \quad w_k \sim \mathcal{N}(0, Q), \quad (6.11)$$

$$y_k = Cx_k + v_k, \quad v_k \sim \mathcal{N}(0, R), \quad (6.12)$$

where  $w_k$  and  $v_k$  are mutually independent Gaussian sequences. The goal is to compute the conditional distribution of  $x_k$  given all measurements  $y_0, \dots, y_k$ ; under the linear-Gaussian assumptions this distribution remains Gaussian and is characterized completely by its mean  $\hat{x}_{k|k}$  and covariance  $P_{k|k}$ .

### 6.7.1 Projection Derivation

Assume we know  $\hat{x}_{k|k}$  and  $P_{k|k}$  at time  $k$ . The one-step-ahead state prediction follows from taking expectations of (6.11):

$$\hat{x}_{k+1|k} = A\hat{x}_{k|k} + Bu_k, \quad (6.13)$$

$$P_{k+1|k} = AP_{k|k}A^\top + Q, \quad (6.14)$$

where  $P_{k+1|k}$  is the a priori covariance. When the  $(k+1)$ -st measurement arrives, we correct the prediction by minimizing the mean-square estimation error:

$$\hat{x}_{k+1|k+1} = \hat{x}_{k+1|k} + K_{k+1}(y_{k+1} - C\hat{x}_{k+1|k}).$$

Choosing  $K_{k+1}$  to minimize  $\text{tr } P_{k+1|k+1}$  yields the familiar gain expression

$$K_{k+1} = P_{k+1|k}C^\top(CP_{k+1|k}C^\top + R)^{-1}, \quad (6.15)$$

and the updated covariance

$$P_{k+1|k+1} = (I - K_{k+1}C)P_{k+1|k}. \quad (6.16)$$

Equations (6.11)–(6.16) constitute the discrete Kalman filter. Appendix A contains the full derivation using conditional expectations and the orthogonality principle for learners interested in the rigorous proof.

### 6.7.2 Innovation Interpretation

Define the innovation (or residual)

$$r_{k+1} = y_{k+1} - C\hat{x}_{k+1|k}.$$

It represents the new information brought by the latest measurement. Its covariance is

$$S_{k+1} = CP_{k+1|k}C^\top + R,$$

which appears in (6.15). Normalizing the innovations via

$$\nu_{k+1} = r_{k+1}^\top S_{k+1}^{-1} r_{k+1}$$

provides a diagnostic statistic: for a correctly tuned filter,  $\nu_{k+1}$  follows a  $\chi^2$  distribution with degrees of freedom equal to the measurement dimension. Systematically large values indicate under-modelled process noise ( $Q$  too small), while systematically small values imply the opposite.

### 6.7.3 Steady-State Filter and Riccati Link

If  $(A, C)$  is detectable and  $(A, Q^{1/2})$  is stabilizable, the Riccati difference equation converges to a positive-definite solution  $P$  satisfying

$$P = APA^\top + Q - APC^\top(CPC^\top + R)^{-1}CPA^\top. \quad (6.17)$$

Equating  $P_{k+1|k} = P$  and  $P_{k+1|k+1} = P$  in (6.16) recovers the discrete algebraic Riccati equation, underscoring the duality between optimal control and optimal estimation. The steady-state Kalman gain becomes

$$K = PC^\top(CPC^\top + R)^{-1},$$

mirroring the LQR gain  $K = (B^\top PB + R)^{-1}B^\top PA$  from the previous section.

### 6.7.4 Practical Tuning Workflow

1. **Scale the state.** Ensure each state component has comparable units; otherwise  $Q$  requires manual rescaling.
2. **Choose baseline covariances.** Start with  $Q = qI$  where  $q$  reflects typical unmodelled accelerations or torques, and set  $R$  using the squared sensor standard deviation.
3. **Run the filter and log innovations.** Use the built-in logging hook (see below) to capture  $(t, r_k, S_k)$  pairs.
4. **Diagnose.** If  $\nu_k$  (normalized innovation squared) consistently exceeds the theoretical expectation, increase the corresponding entries in  $Q$ . If  $\nu_k$  is too small, decrease  $Q$  or increase  $R$ .
5. **Validate closed-loop performance.** Re-run the coupled LQR–Kalman example and examine actuator usage and tracking error.

The experiment harness exposes the innovations automatically when invoked with logging enabled:

```
from scripts.experiments import EXPERIMENTS
result = EXPERIMENTS["kf_lqr"].runner(return_logs=True)
innovation_log = result.metadata["innovation_log"]
t, innov, Sk = innovation_log.T # unpack time series
```

Plot  $|\text{innov}|$  against  $\sqrt{S_k}$  to visualize whether the residuals stay within expected bounds.

### 6.7.5 Case Study 1: Position-Only Sensing

Consider the mass–spring–damper with position measurement only. Select

$$Q = \text{diag}(10^{-4}, 5 \times 10^{-5}), \quad R = 4 \times 10^{-4}.$$

Execute

```
python demo.py kf_lqr --no-plot
```

and observe:

- convergence of the Kalman gain (velocity reconstruction emerges automatically);
- printed performance metrics showing zero steady-state error when combined with the LQR controller;



- innovation statistics that should match the chosen  $Q$ ,  $R$  if the tuning is consistent.

Increase  $Q$  by a factor of ten and rerun the experiment. The estimator responds faster but injects more noise into the control effort—visible in `figures/modern_lqr_controls.png` after regenerating the tutorial plots. Conversely, reducing  $Q$  slows the estimator yet yields smoother control.

### 6.7.6 Case Study 2: Sensor Fusion with Bias

Extend the model to include a sensor bias  $b_k$  by augmenting the state:

$$x_k^{\text{aug}} = \begin{bmatrix} x_k \\ b_k \end{bmatrix}, \quad b_{k+1} = b_k + \eta_k, \quad \eta_k \sim \mathcal{N}(0, \sigma_b^2).$$

Modify `scripts/experiments.py` accordingly (see comments near the Kalman filter setup) and run the simulation with  $\sigma_b = 5 \times 10^{-5}$ . The augmented filter learns the bias online, reducing steady-state error without sacrificing noise rejection. Compare the logged innovations before and after augmentation to verify that the residual statistics return to nominal levels.

### 6.7.7 Case Study 3: Missing Data and Outliers

Kalman filters handle occasional sensor dropouts by skipping the update step. Modify the controller loop to randomly skip updates with probability  $p = 0.1$ :

```
if np.random.rand() < 0.1:
    last_control = sf.step(kf.xhat, reference) # prediction only
else:
    innovation = kf.update(measurement)
    last_control = sf.step(kf.xhat, reference)
```

Inspect the innovation log to ensure the covariance  $P$  inflates during dropouts, reflecting increased uncertainty. Re-enable the update step when measurements resume and confirm the estimator smoothly re-converges. This exercise illustrates how the Kalman framework naturally extends to imperfect sensing environments, a common scenario in embedded control.

## 6.8 Integrated Example: LQR with Kalman Filter

### What?

Combines optimal control and estimation into a full-state-feedback loop using the separation principle.

### Why?

Demonstrates how controller and observer interact in practice, preparing you for implementation.

### When?

After designing  $K$  and Kalman gains separately, as a final integration step.

### How?

Discretize the plant, compute LQR and Kalman gains, and simulate the coupled loop.

### Example

Execute the provided experiment sequence to see tracking performance and estimator convergence for the mass–spring–damper.

Run

```
python demo.py kf_lqr
```

which performs the following steps:

1. Discretize the mass–spring–damper system with sampling time  $T_s = 0.01$  s.
2. Solve the DARE with  $Q = \text{diag}(150, 10)$ ,  $R = 0.4$  to obtain  $K$ .
3. Compute the prefilter  $N_{\text{bar}}$  for step tracking.
4. Run the Kalman filter with  $Q_n = \text{diag}(10^{-4}, 10^{-3})$  and  $R_n = 10^{-4}$ .

Metrics such as steady-state error and overshoot are reported; compare them with the pure state-feedback case to appreciate estimator benefits.

## 6.9 Tutorial: LQR and Kalman Practice

### What?

Guided scripting workflow that regenerates figures and metrics for LQR and Kalman designs.

### Why?

Reinforces theory with reproducible plots and logs, making it easy to experiment with weighting matrices.

### When?

After understanding the algorithms, as hands-on reinforcement or report preparation.

### How?

Run `python -m scripts.tutorials modern --output-dir figures`, inspect responses, and adjust  $Q$ ,  $R$ ,  $Q_n$ ,  $R_n$ .

### Example

Compare `figures/modern_lqr_responses.png` for “tracking” vs. “energy” weightings to see the effect of  $Q/R$  scaling.

1. Generate the modern-control figures with  
`python -m scripts.tutorials modern --output-dir figures`.
2. Use `figures/modern_lqr_responses.png` to compare “tracking” and “energy” weightings in the style suggested by Ljung [5]; note how  $Q$  and  $R$  choices reshape the transient.
3. Inspect `figures/modern_lqr_controls.png` to quantify the actuator effort required by each design and relate it to the penalty matrices.
4. Study `figures/modern_kalman_tracking.png` to see the Kalman estimate converge toward the true trajectory despite measurement noise; experiment with  $Q_n$ ,  $R_n$  inside `tutorial_modern` to observe innovation statistics.

Exact commands and build automation tips are consolidated in Appendix B for quick reference while you work through these exercises.

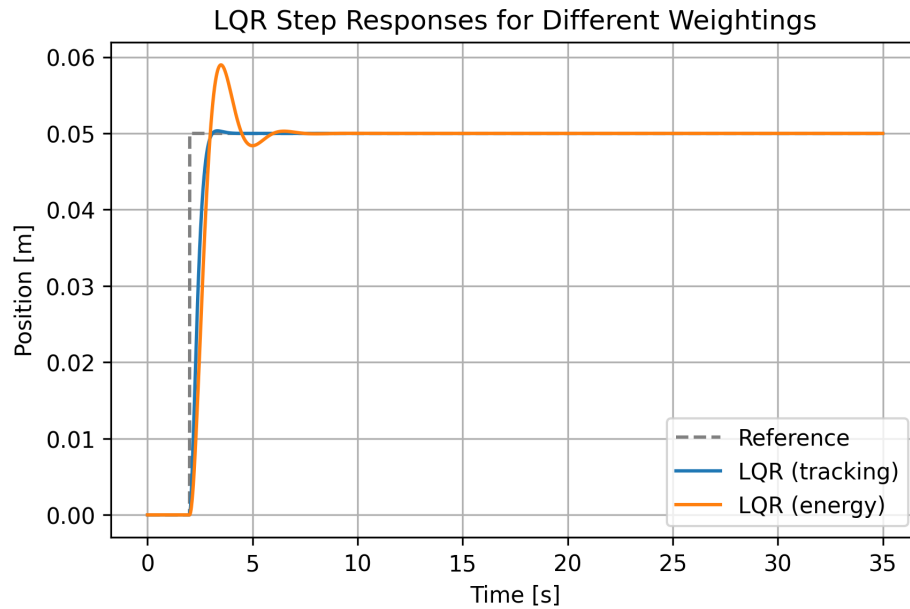


Figure 6.1: LQR responses for different weightings produced by `python -m scripts.tutorials modern`.

## 6.10 Design Trade-Offs

### What?

Discussion of how weighting choices, estimator tuning, and actuator limits interplay.

### Why?

Understanding trade-offs prevents over-aggressive designs and highlights practical constraints.

### When?

During iterative tuning and before deploying on hardware.

### How?

Evaluate how changes in  $Q$ ,  $R$ ,  $Q_n$ ,  $R_n$  impact transient metrics, control effort, and innovation statistics.

### Example

Increase  $Q$  entries in the tutorial script, observe reduced settling time but higher control effort, and decide acceptable compromises.

- **Weight selection:** Large  $Q$  entries can yield aggressive state regulation but may demand high control effort, risking actuator saturation. Balance with  $R$ .
- **Estimator tuning:** Underestimating measurement noise  $R$  leads to noisy control inputs; overestimating  $Q$  yields sluggish response. Use innovation analysis to refine values.
- **Reference tracking:** For non-zero steady-state goals, augment the system with integral action or design feedforward prefilters.

### Key Takeaways

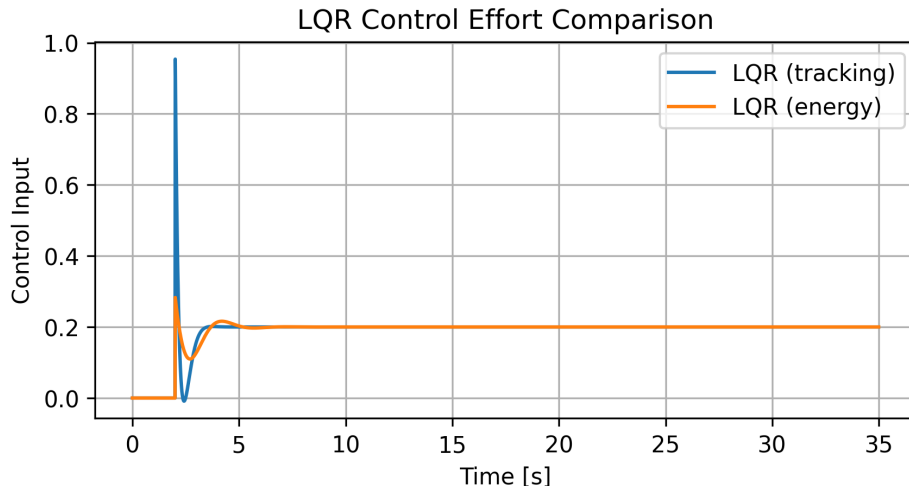


Figure 6.2: Control effort comparison for alternative LQR designs.

- Controllability and observability form the foundation of modern control; verify them before attempting advanced designs.
- LQR provides a principled way to balance performance and effort, while prefilters ensure accurate tracking.
- Kalman filtering enables high-performance control despite sensor noise, completing the state-estimation feedback loop.

## 6.11 Module Review

- **Key tests:** controllability matrix  $\mathcal{C}$  and observability matrix  $\mathcal{O}$  must be full rank before designing feedback or estimators.
- **LQR summary:** solve the DARE for  $P$ , compute  $K = (B^T P B + R)^{-1} B^T P A$ , and use  $N_{\text{bar}}$  for steady-state accuracy; interpret  $Q, R$  weightings with the tutorial overlay in `figures/modern_lqr_responses.png`.
- **Kalman filter:** tune  $Q_n, R_n$  using innovation statistics; verify estimator response with Figure 7.1 and `figures/modern_kalman_tracking.png`.
- **Practice:** regenerate `figures/modern_lqr_responses.png` and modify  $Q, R$  in `tutorial_modern` to see trade-offs described in Ljung [5].
- **Reading:** Ljung [5] Ch. 9–11 for identification-enabled state-space design and Åström and Hägglund [4] for practical loop-shaping comparisons.

## 6.12 Keyword Review

### Keywords

- **State:** minimal set of internal variables required to determine future evolution with the input.

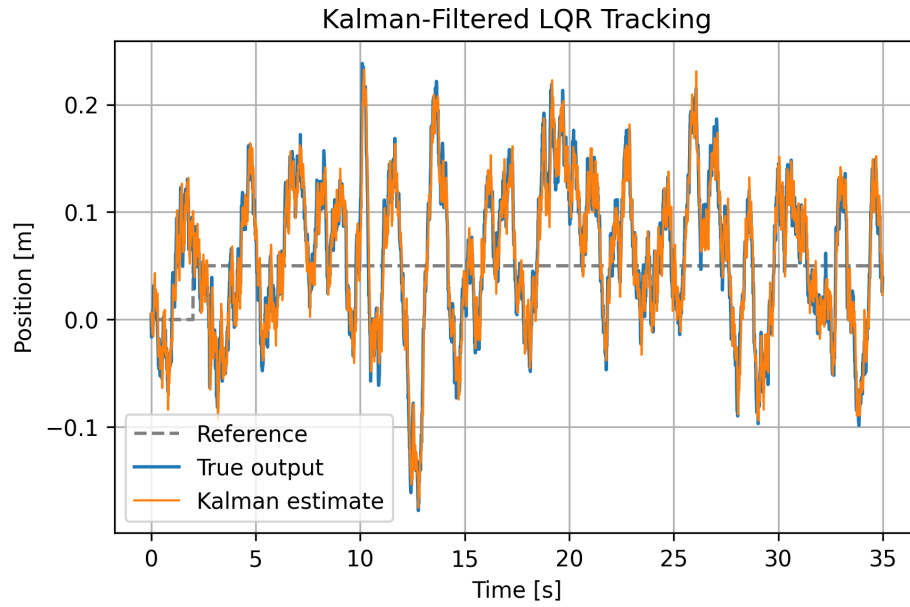


Figure 6.3: Kalman-filtered tracking result generated by the modern tutorial script.

- **Controllability:** ability to reach any state via inputs; captured by the rank of  $\mathcal{C}$ .
- **Observability:** ability to reconstruct the state from outputs; captured by the rank of  $\mathcal{O}$ .
- **LQR:** optimal state-feedback law obtained by solving a Riccati equation to balance error and control effort.
- **Kalman gain:**  $K = PC^T(CPC^T + R)^{-1}$ , weighting how much the estimate trusts new measurements.
- **Separation principle:** controller and observer can be designed independently when the system is controllable and observable.



## Chapter 7

# Estimation and Adaptive Control

### Learning Objectives

- Design observers and Kalman filters to reconstruct unmeasured states and reject sensor noise.
- Analyze innovation statistics to tune stochastic estimators.
- Explore model reference adaptive control (MRAC) through explicit update laws and simulation case studies.

## 7.1 Concept Map for Beginners

### Concept Map

1. Understand deterministic observers first—what they estimate, how fast they converge—and only then add noise to motivate Kalman filtering.
2. Use innovation statistics as a feedback loop for your estimator, just as controller outputs are checked against performance metrics.
3. Extend the estimation mindset to adaptive control: parameters become “states” we estimate online to keep performance robust.

## 7.2 Concept–Math–Engineering Loop

## 7.3 Deterministic State Observers

### What?

Luenberger observers that reconstruct full state vectors using known dynamics and measurements.

### Why?

Many systems lack direct state sensing; observers provide the feedback signals controllers require.

### When?

When sensors measure only a subset of states but the system is observable.

| Concept          | Mathematics                              | Engineering Practice                                                                                   |
|------------------|------------------------------------------|--------------------------------------------------------------------------------------------------------|
| Observer design  | Pole placement of $A - LC$               | Use <code>place_poles</code> and simulation hooks to confirm convergence with noisy measurements.      |
| Kalman filtering | Riccati recursion, innovation covariance | Log $(t, r_k, S_k)$ from <code>EXPERIMENTS["kf_lqr"]</code> to tune $Q, R$ empirically.                |
| Adaptive control | Lyapunov analysis, MIT rule              | Run <code>demo.py mrac</code> and tutorial plots to visualise parameter learning against design goals. |

Table 7.1: Estimation and adaptation concepts tied to their mathematical tools and executable experiments.

### How?

Select observer poles, compute gain  $L$ , and simulate error dynamics to verify convergence speed.

### Example

Place observer poles five times faster than closed-loop poles using `place_poles` and inspect error decay in simulation.

#### 7.3.1 Derivation

For partially measured systems, the discrete Luenberger observer

$$\hat{x}_{k+1} = A\hat{x}_k + Bu_k + L(y_k - C\hat{x}_k)$$

reconstructs the state. The error  $e_k = x_k - \hat{x}_k$  evolves as  $e_{k+1} = (A - LC)e_k$ . Choose  $L$  such that the eigenvalues of  $A - LC$  lie well inside the unit circle (typically 2–5 times faster than the closed-loop poles).

#### 7.3.2 Design Steps

1. Specify desired observer poles  $\Lambda_{\text{obs}}$ .
2. Form the dual system  $(A^\top, C^\top)$  and compute  $L^\top$  via pole placement (e.g., `scipy.signal.place_poles`).
3. Simulate the observer with noisy measurements to ensure convergence speed and noise rejection align with expectations.

#### 7.3.3 Code Mapping

### 7.4 Kalman Filtering

#### What?

Probabilistic state estimation that minimises mean-square error under Gaussian noise assumptions.



| Python                                                                      | Formula                 | Explanation                                     |
|-----------------------------------------------------------------------------|-------------------------|-------------------------------------------------|
| <code>scipy.signal.place_poles</code><br>(A.T, C.T,<br>poles).gain_matrix.T | $L$                     | Generates the observer gain from desired poles. |
| <code>simulate_discrete_system</code><br>with observer-in-the-loop          | $e_{k+1} = (A - LC)e_k$ | Validates convergence numerically.              |

Table 7.2: Luenberger observer implementation.

**Why?**

It weights model predictions and measurements optimally, delivering robust estimates and confidence bounds.

**When?**

When process and measurement noise are significant or uncertainty quantification is needed.

**How?**

Propagate  $(\hat{x}, P)$  through prediction/correction steps and tune  $Q, R$  using innovation statistics.

**Example**

Run `EXPERIMENTS["kf_lqr"].runner(return_logs=True)` to log innovations and adjust  $Q$  until the normalised innovation squared averages near the measurement dimension.

**7.4.1 Stochastic Model**

Assume

$$x_{k+1} = Ax_k + Bu_k + w_k, \quad w_k \sim \mathcal{N}(0, Q), \quad (7.1)$$

$$y_k = Cx_k + v_k, \quad v_k \sim \mathcal{N}(0, R), \quad (7.2)$$

with mutually independent noises. The Kalman recursion is

$$\begin{aligned} \hat{x}_{k|k-1} &= A\hat{x}_{k-1|k-1} + Bu_{k-1}, \\ P_{k|k-1} &= AP_{k-1|k-1}A^\top + Q, \\ K_k &= P_{k|k-1}C^\top(CP_{k|k-1}C^\top + R)^{-1}, \\ \hat{x}_{k|k} &= \hat{x}_{k|k-1} + K_k(y_k - C\hat{x}_{k|k-1}), \\ P_{k|k} &= (I - K_kC)P_{k|k-1}. \end{aligned}$$

The innovation  $r_k = y_k - C\hat{x}_{k|k-1}$  has covariance  $S_k = CP_{k|k-1}C^\top + R$  and serves as a diagnostic statistic.

**7.4.2 Tuning Guidelines**

- Increase  $Q$  to trust the model less and rely more on measurements (useful when unmodeled disturbances dominate).
- Increase  $R$  to trust the measurements less (useful when sensors are noisy).
- Monitor the normalized innovation squared (NIS) statistic  $\tilde{y}_k^T S_k^{-1} \tilde{y}_k$ ; its average should approach the measurement dimension.

### 7.4.3 Code Mapping and Logging

| Python                 | Formula                      | Explanation                                                                                                                                |
|------------------------|------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <code>predict()</code> | $\hat{x}_{k k-1}, P_{k k-1}$ | Method of the filter object; propagates the state and covariance.                                                                          |
| <code>update()</code>  | $K_k, r_k, P_{k k}$          | Applies the correction step and returns the innovation together with the updated covariance.                                               |
| <code>runner()</code>  | $(t, r_k, S_k)$              | Invoke on <code>EXPERIMENTS["kf_lqr"]</code> with <code>return_logs=True</code> to capture the innovation statistics used to tune $Q, R$ . |

Table 7.3: Kalman filter code correspondence.

### 7.4.4 Case Study

Select  $Q = \text{diag}(10^{-4}, 5 \times 10^{-5})$ ,  $R = 4 \times 10^{-4}$  for the mass–spring–damper with position measurements only. Running `python demo.py kf_lqr --no-plot` yields steady-state gains and innovation statistics. Adjust  $Q$  by an order of magnitude to see the trade-off between responsiveness and noise amplification; inspect the logged  $S_k$  to verify tuning.

## 7.5 Model Reference Adaptive Control

### What?

Adaptive laws that adjust controller parameters online so the plant follows a reference model.

### Why?

MRAC counters parameter drift and unmodelled dynamics without manual retuning.

### When?

After ensuring persistent excitation and when plant parameters are uncertain or changing.

### How?

Define the reference model, choose an adaptation law (e.g., MIT rule), and monitor Lyapunov-based error metrics.

### Example

Execute `python demo.py mrac` to observe  $\theta_k$  convergence and tracking error shrinking toward the reference model output.

### 7.5.1 Derivation

MRAC enforces  $y_k \rightarrow y_{m,k}$  where the reference model satisfies

$$y_{m,k+1} = a_m y_{m,k} + b_m r_k. \quad (7.3)$$

For the first-order plant  $y_{k+1} = a y_k + b u_k$ , use the control law

$$u_k = \theta_1 y_k + \theta_2 r_k \quad (7.4)$$

with parameter update via the MIT rule

$$\theta_{k+1} = \theta_k - \gamma \phi_k e_k, \quad (7.5)$$

where  $\phi_k = [y_k \ r_k]^\top$  and  $e_k = y_k - y_{m,k}$ . Define  $J_k = \frac{1}{2}e_k^2$ ; the gradient step

$$\theta_{k+1} = \theta_k - \gamma \frac{\partial J_k}{\partial \theta} = \theta_k - \gamma \left( \frac{\partial e_k}{\partial \theta} \right) e_k$$

reduces to the MIT rule because  $\partial e_k / \partial \theta = \phi_k$ .

### 7.5.2 Stability Insight

Define a Lyapunov function candidate

$$V = \frac{1}{2}e_k^2 + \frac{1}{2\gamma}(\theta_k - \theta^*)^T(\theta_k - \theta^*), \quad (7.6)$$

where  $\theta^*$  denotes ideal parameters that achieve perfect tracking. Under standard MRAC assumptions (matching conditions, persistent excitation), the adaptation law ensures  $V$  decreases, leading to bounded tracking error.

### 7.5.3 Code Mapping

| Python                             | Formula                    | Explanation                                             |
|------------------------------------|----------------------------|---------------------------------------------------------|
| <code>MRACController.step()</code> | $u_k = \theta^\top \phi_k$ | Applies the adaptive control law.                       |
| <code>MRACController.theta</code>  | $\theta_k$                 | Stores parameter estimates; log to analyse convergence. |

Table 7.4: MRAC algorithm in `scripts/controllers.py`.

### 7.5.4 Simulation Practice

Run `python demo.py mrac`. The experiment discretizes a first-order plant with  $a = -0.4$ ,  $b = 1$ , uses  $a_m = 0.97$ ,  $b_m = 0.03$ , and  $\gamma = 0.2$ . Record steady-state error, overshoot, and parameter evolution. Adjust  $\gamma$  to explore convergence versus noise sensitivity; add parameter logging to visualise  $\theta_k$  settling.

## 7.6 Tutorial: Adaptive Control Walkthrough

### What?

A scripted tour through adaptive tracking and parameter convergence figures.

### Why?

Visualising trajectories helps interpret adaptation speed and robustness to noise.

### When?

After learning the equations, as a sandbox for experimenting with adaptation gains.

**How?**

Run `python -m scripts.tutorials adaptive --output-dir figures` and inspect tracking and parameter plots while varying  $\gamma$ .

**Example**

Increase  $\gamma$  to 0.4, rerun the tutorial, and note faster convergence but amplified parameter oscillations.

1. Produce the adaptive-control figures with  
`python -m scripts.tutorials adaptive --output-dir figures`.
2. Inspect Figure 7.1 to see how the plant output matches the reference model; relate the transient to the Lyapunov design insights in Krstić et al. [6].
3. Examine Figure 7.2 to monitor the evolution of  $\theta_1$  and  $\theta_2$ . Adjust the adaptation gain  $\gamma$  inside `tutorial_adaptive` to visualise the trade-off between convergence speed and robustness.

Appendix B lists the supported `demo.py` experiments and plotting options you can reuse while exploring these scenarios.

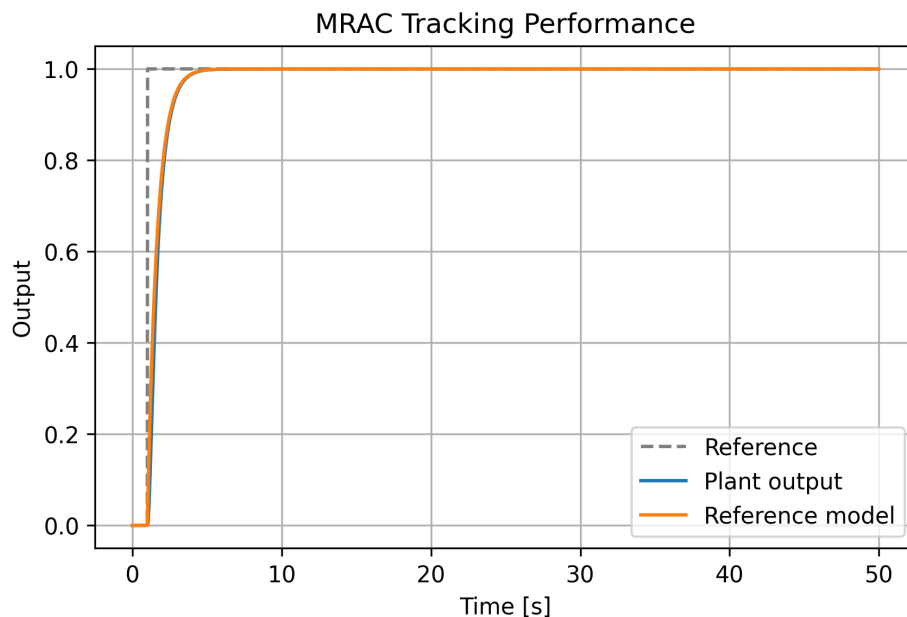


Figure 7.1: MRAC tracking performance generated by `python -m scripts.tutorials adaptive`.

## 7.7 Handling Missing or Delayed Measurements

**What?**

Strategies for keeping estimators stable under packet drops, outliers, or sensor latency.

**Why?**

Real-world systems rarely deliver perfect data; robustness to measurement issues is essential.

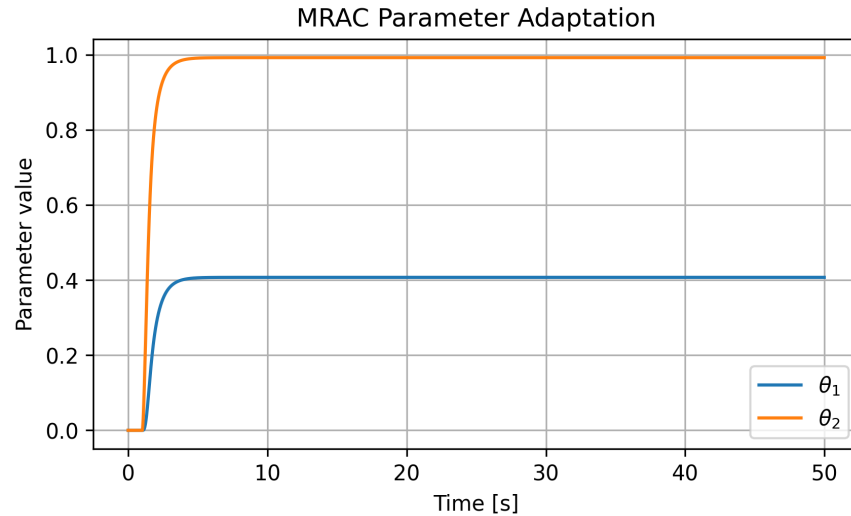


Figure 7.2: Parameter adaptation trajectories illustrating online learning.

#### When?

During estimator deployment, especially over networks or with low-cost sensors.

#### How?

Propagate without updates during dropouts, inflate covariance, and gate outliers.

#### Example

Modify `simulate_discrete_system` to drop every fifth measurement and observe how covariance inflation maintains state estimate accuracy.

Modify `simulate_discrete_system` to include packet drops or sensor delays. Strategies to maintain estimator performance include:

- **Prediction during dropouts:** propagate the estimate using the process model when measurements are unavailable.
- **Covariance inflation:** increase  $P$  during prolonged dropouts to reflect growing uncertainty.
- **Outlier rejection:** ignore innovations exceeding a specified threshold to handle bad data.

## 7.8 Hands-On Practice

#### What?

Exercises that apply observers, Kalman filters, and adaptive controllers to scripted experiments.

#### Why?

Practice reveals tuning nuances and builds confidence before tackling custom plants.

#### When?

After studying the theory—use the tasks as immediate reinforcement.

**How?**

Run the experiment harness, log results, adjust gains/noise levels, and compare against analytical expectations.

**Example**

Toggle measurement dropouts while plotting innovations to see how covariance inflation affects the Kalman filter's residual statistics.

1. Design a Luenberger observer using desired poles; verify  $A - LC$  eigenvalues numerically.
2. Tune the Kalman filter by logging innovations:

```
result = EXPERIMENTS["kf_lqr"].runner(return_logs=True)
innovation_log = result.metadata["innovation_log"]
```

Adjust  $Q$ ,  $R$  until the residual statistics match the theoretical covariance.

3. Use the MRAC experiment to visualise parameter convergence for different  $\gamma$  values; relate overshoot changes to the Lyapunov analysis.
4. Introduce measurement dropouts and confirm covariance inflation keeps the estimator responsive.

**Key Takeaways**

- Observers and Kalman filters reconstruct hidden states and mitigate sensor noise; tune them using innovation statistics.
- Adaptive control compensates for parameter drift but must be monitored to avoid instability.
- Combined estimation and adaptation enable robust performance in the face of uncertainty and measurement imperfections.

**7.9 Module Review**

- **Observer essentials:** choose observer poles (or gains) at least twice the closed-loop bandwidth, use Lyapunov criteria to confirm convergence.
- **Kalman tuning:** base  $Q$ ,  $R$  on physical insight and refine via innovation statistics.
- **Adaptive laws:** MIT update  $\theta_{k+1} = \theta_k - \gamma \phi_k e_k$  balances tracking accuracy and robustness; monitor parameter trajectories.
- **Practice:** run `python -m scripts.tutorials adaptive` to explore noise, dropout, and gain effects.
- **Reading:** Krstić et al. [6] for rigorous adaptive proofs and Ljung [5] for identification-integrated estimation.

## 7.10 Keyword Review

### Keywords

- **Luenberger observer**: deterministic estimator with error dynamics  $A - LC$  tuned via pole placement.
- **Innovation**: difference  $r_k = y_k - C\hat{x}_{k|k-1}$  summarising new measurement information.
- **Kalman gain**: optimal weighting  $K_k = P_{k|k-1}C^\top(CP_{k|k-1}C^\top + R)^{-1}$  balancing model and measurements.
- **Normalized innovation squared (NIS)**: statistic  $r_k^\top S_k^{-1} r_k$  used to diagnose filter tuning.
- **Model reference adaptive control (MRAC)**: scheme forcing plant output toward a reference model via online parameter updates.
- **MIT rule**: gradient-descent update  $\theta_{k+1} = \theta_k - \gamma \phi_k e_k$  minimizing tracking error.

For motivated readers, Krstić et al. [6] expand these ideas to nonlinear plants, while Ljung [5] show how estimation theory integrates with system identification workflows.





## Chapter 8

# Digital Implementation and Practical Considerations

### Learning Objectives

- Understand how sampling, quantization, and computational delays affect closed-loop performance.
- Translate continuous-time controllers into numerically stable discrete implementations.
- Plan validation workflows from simulation to hardware-in-the-loop testing.

## 8.1 Concept Map for Beginners

### Concept Map

1. Look at sampling as the act of taking snapshots; relate Nyquist and aliasing to everyday examples like misinterpreted wagon wheels.
2. Translate controllers carefully so that the discrete implementation preserves the intent of the continuous design.
3. Build confidence stepwise—simulate, run on the processor, then integrate hardware—so each stage validates assumptions about timing and quantization.

## 8.2 Concept–Math–Engineering Loop

## 8.3 Sampling Theory Refresher

### What?

Reviews how sampling, aliasing, and pole mapping affect discrete-time representations.

### Why?

Sampling choices directly impact closed-loop stability and fidelity when migrating to digital controllers.

| Concept                 | Mathematics                                                   | Engineering Practice                                                                         |
|-------------------------|---------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Sampling effects        | Nyquist limit, discrete pole mapping<br>$z = e^{\lambda T_s}$ | Reproduce aliasing plots with<br><code>python -m scripts.tutorials.digital</code> .          |
| Discretization fidelity | Euler and Tustin substitutions                                | Compare discrete PID implementations by switching methods in the tutorial.                   |
| Nonidealities           | Quantization models, delay-induced phase lag                  | Use simulator options for quantization/jitter and inspect generated figures for margin loss. |

Table 8.1: Connecting digital-control concepts to the math and experimentation workflow.

**When?**

Before discretising controllers or choosing hardware update rates.

**How?**

Apply Nyquist limits, map continuous poles via  $z = e^{\lambda T_s}$ , and simulate aliasing scenarios.

**Example**

Use the tutorial to sample a 20 Hz plant at 25 Hz and observe the resulting 5 Hz alias in the output spectrum.

Sampling a continuous signal  $x(t)$  at period  $T_s$  yields  $x[k] = x(kT_s)$ . Key implications:

- Nyquist criterion requires sampling frequency  $\omega_s = 2\pi/T_s$  to exceed twice the highest significant frequency component ( $\omega_s > 2\omega_{\max}$ ).
- Continuous-time poles  $\lambda_i$  map to discrete poles  $z_i = e^{\lambda_i T_s}$ . Poles with large negative real parts may cluster near the origin if  $T_s$  is large, distorting dynamics.
- Input/output delays  $\tau$  translate to factors of  $z^{-d}$  with  $d = \lceil \tau/T_s \rceil$ , reducing phase margin.

**8.3.1 Worked Example: Aliasing and Timing Jitter**

Consider a plant whose dominant mode oscillates at 20 Hz ( $\omega_{\max} = 40\pi$  rad/s). Sampling at  $T_s = 0.04$  s (25 Hz) violates Nyquist because  $\omega_s = 50\pi < 2\omega_{\max}$ . The aliased frequency observed by the controller is

$$f_{\text{alias}} = |f_{\text{signal}} - Nf_s| = |20 - 1 \cdot 25| = 5 \text{ Hz},$$

causing the loop to chase a much slower oscillation and mis-tune gains. Edit `tutorial_digital` to set the sampling frequency to 25 Hz and re-run

```
python -m scripts.tutorials.digital --output-dir figures
```

to regenerate `figures/digital_sampling_spectrum.png` with both the original 20 Hz peak and the aliased 5 Hz component.

Execution jitter produces an equivalent phase lag. If a task scheduled for  $T_s = 10$  ms experiences a worst-case delay of  $\delta = 2$  ms, the additional phase lag near crossover frequency  $\omega_c$  is approximately  $\Delta\phi \approx -\omega_c\delta$ . For  $\omega_c = 30$  rad/s, the lag reaches  $-3.4^\circ$ . Extend `tutorial_digital` to draw a random delay (e.g., uniform in  $[0, \delta]$ ) before each controller evaluation and log the resulting margins so that `figures/digital_jitter_margin.png` captures the degradation.

## 8.4 Controller Discretization

### What?

Methods for converting continuous controllers into stable, accurate discrete implementations.

### Why?

Naïve discretisation can destabilise loops; understanding mapping choices preserves design intent.

### When?

After designing continuous compensators and before coding them in embedded firmware.

### How?

Choose an  $s \mapsto z$  mapping (forward/backward Euler, Tustin), derive discrete transfer functions, and test numerically.

### Example

Discretise a lead compensator with Tustin and compare with the exact ZOH discretisation produced by SciPy's `cont2discrete`.

Common  $s \mapsto z$  mappings:

$$s \approx \frac{z - 1}{T_s} \quad (\text{forward Euler}), \quad (8.1)$$

$$s \approx \frac{z - 1}{z T_s} \quad (\text{backward Euler}), \quad (8.2)$$

$$s \approx \frac{2}{T_s} \frac{z - 1}{z + 1} \quad (\text{Tustin/bilinear}). \quad (8.3)$$

Forward Euler is simple but may destabilize high-frequency dynamics; Tustin preserves stability and usually offers better fidelity. Always implement derivative terms with low-pass filtering to avoid amplifying noise.

## 8.5 Quantization, Saturation, and Noise

### What?

Practical nonidealities introduced by finite resolution and actuator limits.

### Why?

These effects can erode performance or cause instability if ignored during design.

### When?

During controller implementation and hardware testing.

### How?

Model quantisation as rounding, include saturation blocks, and simulate noise to evaluate robustness.

### Example

Run the simulator with `quantization=1e-4` and actuator limits to see how PID integral windup appears without anti-windup logic.

Finite resolution introduces quantization noise. Simulate the effects with the harness:

Listing 8.1: Quantization impact

```
result = simulate_discrete_system(
    Ad, Bd, C,
    controller_step,
    ts=Ts,
    steps=3000,
    reference=lambda k: 0.05 if k > 50 else 0.0,
    measurement_noise_std=5e-4,
    quantization=1e-4,
    actuator_limits=(-5.0, 5.0),
)
print(result.performance_metrics())
```

Integral action must incorporate anti-windup logic to prevent integrator windup when actuators saturate. Options include integral state clamping or back-calculation.

### 8.5.1 Code Mapping

| Python                            | Formula                                              | Explanation                                                                                             |
|-----------------------------------|------------------------------------------------------|---------------------------------------------------------------------------------------------------------|
| Simulator option                  | $y_k^{\text{meas}} = \text{round}(y_k/\Delta)\Delta$ | Uses the quantization flag in <code>simulate_discrete_system</code> to enforce the step size $\Delta$ . |
| <code>PIDController.step()</code> | Anti-windup logic                                    | Includes saturation checks and integral clamping/back-calculation.                                      |

Table 8.2: Digital implementation effects mapped to simulation code.

## 8.6 Tutorial: Digital Control Experimentation

### What?

Scripted experiments exploring sampling, delay, and quantisation effects on PID loops.

### Why?

Hands-on trials highlight the performance penalties of implementation constraints.

### When?

Before deploying code to hardware or when documenting digital design trade-offs.

### How?

Execute `python -m scripts.tutorials digital --output-dir figures` and compare ideal versus nonideal scenarios.

### Example

Generate `figures/digital_pid_responses.png` to compare nominal and delayed executions, noting increased oscillations due to jitter.

1. Generate the digital-control figures with  
`python -m scripts.tutorials digital --output-dir figures.`

2. Study Figure 8.1 to compare ideal, quantized, and delayed PID loops; note how quantization amplifies steady-state error while delay introduces oscillation.
3. Examine Figure 8.3 to see how different sampling periods reshape the output spectrum, echoing the sampling guidelines of Åström and Hägglund [4].
4. Inspect Figure 8.2 to assess actuator usage under each scenario and motivate anti-windup strategies.

Command-line recipes for rebuilding these figures and compiling the notes are summarised in Appendix B.

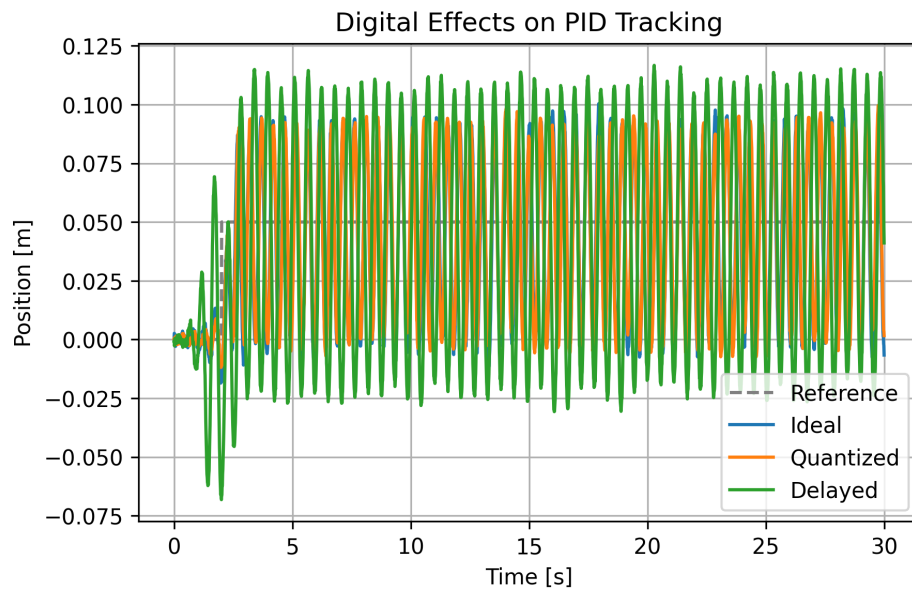


Figure 8.1: Impact of quantization and sensor delay on PID tracking.

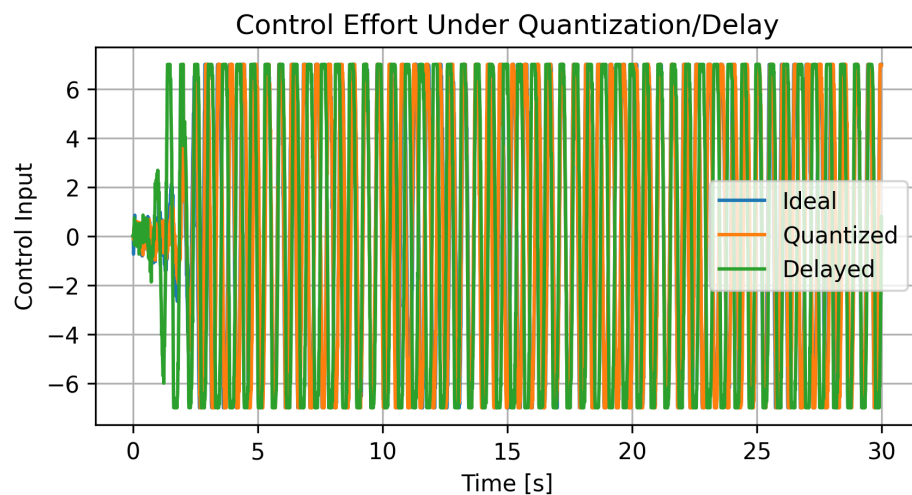


Figure 8.2: Control effort under ideal, quantized, and delayed execution.

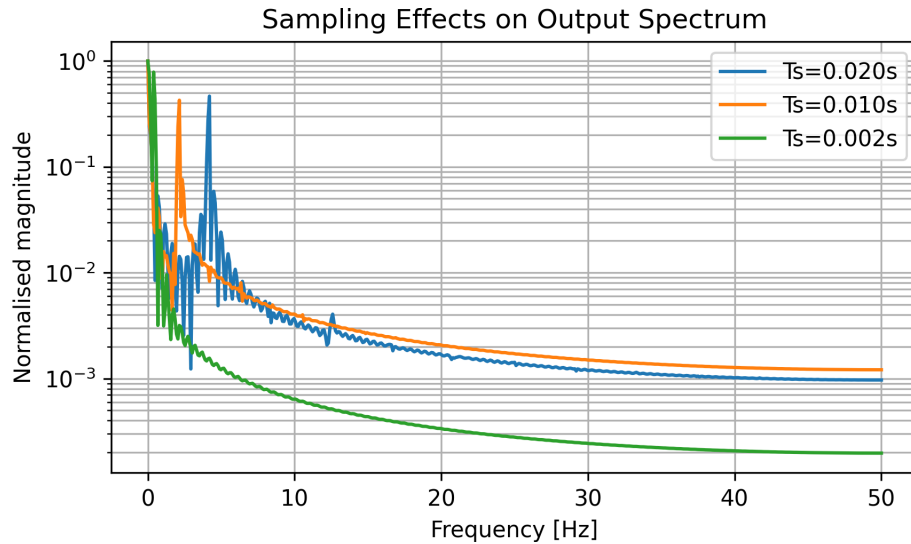


Figure 8.3: Output spectra for multiple sampling periods.

## 8.7 Scheduling and Real-Time Constraints

### What?

Considerations for meeting timing deadlines and mitigating jitter in real-time systems.

### Why?

Missed deadlines or variable execution times translate into effective delays that can destabilise control loops.

### When?

While allocating tasks to processors or designing RTOS schedules.

### How?

Profile computation, budget slack, add watchdogs, and design with extra phase margin to tolerate jitter.

### Example

Estimate phase margin loss using  $\Delta\phi \approx -\omega_c\delta$  for a task with 2 ms jitter at  $\omega_c = 30$  rad/s.

Digital controllers execute on embedded processors with finite resources:

- **Computation time:** the control algorithm must finish within  $T_s$ . Profiling and fixed-point optimization may be necessary on microcontrollers.
- **Jitter:** Variations in execution time act as multiplicative uncertainty. Design with additional phase margin to accommodate jitter.
- **Latency:** Sensor-to-actuator delays reduce effective phase margin; compensate via Smith predictors or redesign with lower bandwidth.

## 8.8 Validation Pipeline

### What?

A staged approach (SIL  $\rightarrow$  PIL  $\rightarrow$  HIL) for verifying controllers before deployment.

### Why?

Structured validation catches issues early, reducing risk during hardware trials.

### When?

After implementing the controller in software and before integration with the physical plant.

### How?

Simulate with software-in-the-loop, execute on target hardware with plant models, then connect to real sensors/actuators.

### Example

Run SIL simulations with quantisation enabled, then execute the same control law on a development board in PIL mode to compare timing.

1. **Software-in-the-loop (SIL)**: run Python simulations that include quantization, saturation, noise, and network-related effects.
2. **Processor-in-the-loop (PIL)**: execute compiled control code on the target processor while interfacing with a simulator to measure timing and numerical precision.
3. **Hardware-in-the-loop (HIL)**: connect actual hardware sensors and actuators (or a real-time plant emulator) to validate end-to-end behavior before deploying to the real system.

Document the validation steps in the appendix, embedding diagnostic plots and logging procedures.

## 8.9 Implementation Checklist

### What?

A quick audit of numerical and safety considerations prior to release.

### Why?

Ensures critical safeguards—overflow protection, watchdogs, reproducible builds—are in place.

### When?

Immediately before deploying firmware or handing off to quality assurance.

### How?

Walk through the checklist items: scaling, anti-windup, watchdog timers, configuration management.

### Example

Confirm integral states are clamped to actuator limits and that configuration files record the exact PID gains shipped.

- Verify numerical robustness (overflow, quantization, scaling) for fixed-point implementations.
- Use watchdog timers and safety interlocks to handle communication failures or saturation events.

- Maintain reproducible build scripts and configuration management so that controller parameters match the documentation.

### Key Takeaways

- Digital implementation details significantly influence real-world performance; treat them as part of the control design process.
- Proper discretization, anti-windup, and scheduling analysis prevent instability caused by sampling and computational effects.
- A staged validation strategy (SIL  $\rightarrow$  PIL  $\rightarrow$  HIL) mitigates risk before deploying controllers to physical hardware.

## 8.10 Module Review

- **Sampling:** check Nyquist ( $\omega_s > 2\omega_{\max}$ ) and preview aliasing via Figure 8.3; adjust  $T_s$  accordingly.
- **Quantisation/delay:** evaluate tracking degradation (Figure 8.1) and control effort (Figure 8.2); implement anti-windup where saturation is visible.
- **Workflow:** follow the SIL  $\rightarrow$  PIL  $\rightarrow$  HIL path; capture logs and reproduce the tutorial to document scenarios.
- **Practice:** run `python -m scripts.tutorials digital` and explore network-induced jitter or sensor noise to build intuition before hardware deployment.
- **Reading:** Åström and Hägglund [4] Ch. 11 for sampled-data issues and Ljung [5] Ch. 12 for implementation case studies.

## 8.11 Keyword Review

### Keywords

- **Sampling period  $T_s$ :** time between controller updates; sets discrete-time dynamics.
- **Aliasing:** misinterpretation of high-frequency signals when  $T_s$  violates Nyquist.
- **Jitter:** variation in execution timing that behaves like additional delay.
- **Quantization:** rounding of signals to discrete levels, introducing effective noise.
- **Anti-windup:** strategy to prevent integrator accumulation during saturation.
- **HIL testing:** hardware-in-the-loop validation combining real controllers with simulated plants.



## Chapter 9

# Learning-Based Control and PDE Adaptive Systems

### Learning Objectives

- Understand foundational ideas in system identification and data-driven modeling for beginners.
- Explore parameter-estimation workflows inspired by Åström and Hägglund [3] and Ljung [5].
- Gain intuition for Lyapunov-based adaptive control of distributed-parameter (PDE) systems following Krstić et al. [6].
- Connect stability theory for nonlinear dynamics with practical scripts and figures.

## 9.1 Concept Map for Beginners

### Concept Map

1. Start from simple data models (e.g., ARX) so every regression weight has a clear physical meaning before tackling richer structures.
2. Use Lyapunov functions as a bridge between learned models and stability guarantees—understand what must stay bounded.
3. Extend your finite-dimensional intuition to PDEs by discretising space, analysing energy, and confirming results with tutorial plots.

## 9.2 Concept–Math–Engineering Loop

## 9.3 Learning Models from Data

### What?

System identification techniques that estimate model parameters from input–output data.

### Why?

Accurate models enable better controllers when first-principles derivations are insufficient.

| Concept                 | Mathematics                                   | Engineering Practice                                                                             |
|-------------------------|-----------------------------------------------|--------------------------------------------------------------------------------------------------|
| Data-driven modelling   | Recursive least squares, regression           | Run <code>tutorial_learning</code> to see parameters converge and evaluate residual variance.    |
| Stability certification | Lyapunov functions, energy decay              | Compute discrete energy $E_k$ in the tutorial to verify positive-definite matrices.              |
| PDE adaptive control    | Spatial discretisation, boundary control laws | Adjust adaptation gain $\gamma$ in the learning tutorial and inspect energy plots for stability. |

Table 9.1: How Module 9 couples identification, stability theory, and numerical experiments.

### When?

When plant parameters are unknown, drifting, or difficult to measure directly.

### How?

Set up regression structures (e.g., ARX, RLS), gather excitation data, and update parameters online.

### Example

Run the RLS tutorial to estimate coefficients of a first-order model and compare predicted vs. measured outputs.

System identification turns measured input–output pairs into predictive models. Ljung [5] presents a unified view spanning ARX, state-space, and nonlinear structures; for beginners we start with the scalar autoregressive model

$$y_{k+1} = ay_k + bu_k + w_k, \quad (9.1)$$

where  $w_k$  captures disturbances. Collecting the regressor  $\phi_k = [y_k \ u_k]^T$  allows the compact form  $y_{k+1} = \phi_k^T \theta + w_k$ ,  $\theta = [a \ b]^T$ .

#### 9.3.1 Recursive Least Squares (RLS)

RLS updates the parameter estimate after each sample:

$$K_k = P_{k-1} \phi_k (\lambda + \phi_k^T P_{k-1} \phi_k)^{-1}, \quad (9.2)$$

$$\theta_k = \theta_{k-1} + K_k (y_{k+1} - \phi_k^T \theta_{k-1}), \quad (9.3)$$

$$P_k = \lambda^{-1} (P_{k-1} - K_k \phi_k^T P_{k-1}), \quad (9.4)$$

with forgetting factor  $\lambda \in (0, 1]$ . RLS converges rapidly and is widely used in industrial autotuning, as discussed by Åström and Hägglund [3].

#### 9.3.2 Code Mapping

#### 9.3.3 Tutorial: Parameter Identification

1. Run `python -m scripts.tutorials learning --output-dir figures` to generate identification and PDE-control figures.

| Python                            | Formula                    | Explanation                                                                    |
|-----------------------------------|----------------------------|--------------------------------------------------------------------------------|
| <code>rls_update()</code>         | $K_k, \theta_k, P_k$       | Part of the learning tutorial module; evaluates the RLS recursion each sample. |
| <code>numpy.var(residuals)</code> | $\text{Var}(w_k)$ estimate | Quantifies disturbance variance for model verification.                        |

Table 9.2: RLS equations aligned with code in `scripts/tutorials`.

2. Inspect Figure 9.1 to see the RLS estimate converge toward the true coefficients while a small measurement noise perturbs the trajectory.
3. Modify the excitation signal or forgetting factor in `tutorial_learning` to observe how richness of data affects convergence.

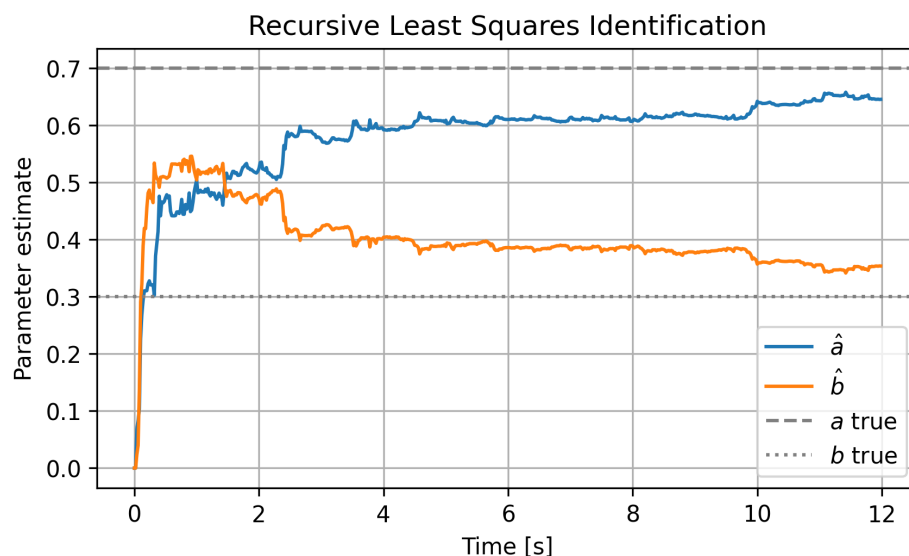


Figure 9.1: Recursive least squares parameter identification inspired by Ljung [5].

## 9.4 Stability Theory Refresher

### What?

Review of Lyapunov methods for both finite- and infinite-dimensional systems.

### Why?

Guarantees that learned or adaptive controllers maintain stability despite parameter updates.

### When?

After fitting data-driven models and prior to deploying adaptive laws.

### How?

Construct Lyapunov functions, evaluate their derivatives, and ensure they decrease along trajectories.

**Example**

Use the energy functional  $E(t)$  for the heat equation and show numerically that adaptive boundary control enforces  $E_{k+1} - E_k < 0$ .

Nonlinear stability hinges on Lyapunov functions: if  $V(x)$  is positive definite and  $\dot{V}(x) \leq 0$ , the origin is stable. Ljung [5] emphasizes combining data-driven identification with stability certificates to validate learned models. In discrete time, choose  $V(x_k) = x_k^T P x_k$ ,  $P \succ 0$ , and verify  $V(x_{k+1}) - V(x_k) \leq 0$ .

## 9.5 Adaptive Control of PDEs

**What?**

Techniques for stabilising distributed-parameter systems via adaptive boundary control.

**Why?**

Many processes (thermal, fluid) obey PDEs; adaptive methods manage parameter uncertainty over space.

**When?**

When lumped-parameter models are inadequate and spatial dynamics matter.

**How?**

Discretise the PDE, design Lyapunov-based adaptation laws, and monitor energy decay.

**Example**

Execute `tutorial_learning` to generate `figures/learning_pde_energy.png` showing Lyapunov energy convergence.

The adaptive boundary-control framework of Krstić et al. [6] shows that infinite-dimensional systems (e.g., the heat equation) can be stabilized via Lyapunov design. Consider the 1-D heat equation on  $[0, 1]$  with boundary input  $u(t)$  applied at  $x = 1$ . A candidate Lyapunov functional is the energy  $E(t) = \int_0^1 x(\xi, t)^2 d\xi$ . Boundary control laws combined with adaptation can force  $E(t)$  to decay despite uncertain plant parameters.

Discretize the spatial domain into  $N$  nodes using spacing  $\Delta\xi = 1/(N - 1)$ . The semi-discrete dynamics become

$$\dot{\mathbf{x}} = \frac{\alpha}{\Delta\xi^2} L\mathbf{x} + Bu + d,$$

where  $L$  is the tridiagonal Laplacian matrix,  $B$  injects the boundary input, and  $d$  lumps model mismatch. The discrete energy  $E_k = \mathbf{x}_k^T W \mathbf{x}_k$  with weighting  $W = \Delta\xi I$  satisfies

$$E_{k+1} - E_k = \mathbf{x}_k^T ((A^T W + W A)) \mathbf{x}_k + 2\Delta t \mathbf{x}_k^T W (Bu_k + d_k),$$

revealing how the control input shapes the energy decay. The tutorial script computes these matrices explicitly so beginners can print  $A^T W + W A$  and confirm it is negative definite once the adaptive law converges.

| Python                            | Formula                | Explanation                                                                                                     |
|-----------------------------------|------------------------|-----------------------------------------------------------------------------------------------------------------|
| <code>build_pde_matrices()</code> | $A, B, W$              | Helper in <code>tutorials.learning</code> that generates the discretized Laplacian and boundary-input matrices. |
| <code>compute_energy()</code>     | $E_k = x_k^\top W x_k$ | Returns the discrete energy so the tutorial can log Lyapunov decay.                                             |

Table 9.3: Mapping the PDE adaptive-control derivation to code.

### 9.5.1 Code Mapping

### 9.5.2 Tutorial: Adaptive PDE Control

1. Using the same `learning` tutorial command generates Figure 9.2. The script discretizes the heat equation, applies adaptive boundary control, and plots energy decay.
2. Experiment with the adaptation gain  $\gamma$  in `tutorial_learning` to observe how convergence speed and stability trade off; relate these observations to the Lyapunov arguments in Krstić et al. [6].
3. Connect the concept back to finite-dimensional Lyapunov theory and observe how energy plots provide immediate stability diagnostics for beginners.

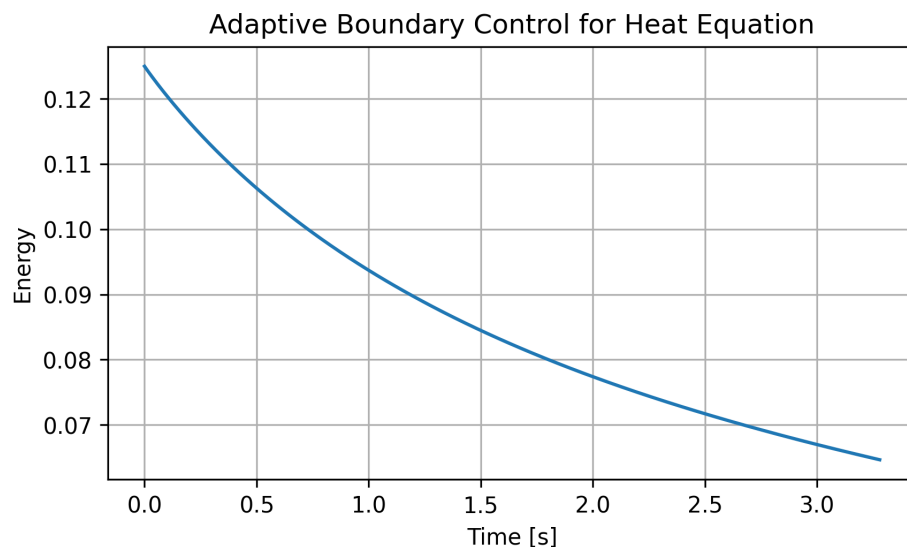


Figure 9.2: Energy decay for an adaptively controlled heat equation (based on concepts from Krstić et al. [6]).

## 9.6 Putting It Together

### What?

Integrates identification, stability analysis, and adaptive control into a cohesive workflow.

**Why?**

Real projects iterate between learning and control; this section highlights the feedback loop between them.

**When?**

When planning experiments that alternate between data collection and controller updates.

**How?**

Follow the measure  $\rightarrow$  learn  $\rightarrow$  verify  $\rightarrow$  control cycle, logging diagnostics at each step.

**Example**

Identify parameters with RLS, design an LQR using those estimates, and validate energy decay with the adaptive PDE controller.

- Use identification to estimate low-order models before designing controllers, following the iterative approach of Åström and Hägglund [3].
- Employ Lyapunov analysis (finite or infinite dimensional) to certify stability of both classical and data-driven controllers.
- Embrace the workflow of measure  $\rightarrow$  learn  $\rightarrow$  verify  $\rightarrow$  control, blending insights from Ljung [5] and Krstić et al. [6] to guide further study.

## 9.7 Practice Checklist

**What?**

Actionable tasks that reinforce data-driven and adaptive concepts.

**Why?**

Practicing each step ensures you can move between identification, control design, and validation smoothly.

**When?**

After reading the chapter and before applying techniques to new datasets.

**How?**

Execute tutorial scripts, vary adaptation gains, and feed identified parameters into downstream controllers.

**Example**

Run `python -m scripts.tutorials learning` with different excitation signals and observe how parameter convergence speed changes.

1. Collect excitation data and run the RLS tutorial; compare estimated parameters with ground truth.
2. Use residual variance to judge whether the linear model captures disturbances adequately.
3. Generate PDE adaptive-control plots and vary the adaptation gain  $\gamma$  to see how energy decay changes.
4. Feed identified parameters into modern-control chapters (e.g., LQR) to close the loop between learning and control synthesis.

Appendix B details the reusable commands for these tutorials so you can re-run them without digging through script docstrings.

## 9.8 Module Review

- **Identification recap:** review the RLS recursion and monitor covariance  $P_k$  to avoid numerical issues; ensure persistent excitation for convergence.
- **PDE insight:** interpret the energy plot (Figure 9.2) as a Lyapunov witness and observe how adaptive boundary control shapes  $E(t)$ .
- **Practice:** rerun `python -m scripts.tutorials learning` and vary  $\gamma$  or the disturbance profile to test robustness.
- **Integration:** feed the identified parameters into LQR/ADRC designs to complete the learn-control loop emphasised in Ljung [5].
- **Further reading:** Krstić et al. [6] for rigorous proofs and Ljung [5] for advanced data-driven case studies.

## 9.9 Keyword Review

### Keywords

- **System identification:** using measured inputs/outputs to estimate model parameters.
- **Recursive least squares (RLS):** online regression algorithm with covariance update  $P_k$  and gain  $K_k$ .
- **Forgetting factor  $\lambda$ :** weighting parameter that balances old versus new data in RLS.
- **Lyapunov functional:** energy-like scalar (or integral) used to certify stability for PDEs and nonlinear systems.
- **Adaptive gain  $\gamma$ :** tuning knob controlling speed/robustness of parameter updates in MRAC/PDE controllers.
- **Energy decay:** decrease of  $E_k = x_k^\top W x_k$  indicating stabilization in discretized PDE systems.





## Chapter 10

# Active Disturbance Rejection Control (ADRC)

### Learning Objectives

- Build a formal understanding of the ADRC architecture: tracking differentiator, extended state observer (ESO), and disturbance-compensating controller.
- Derive the linear ADRC (LADRC) equations for second-order plants and connect the bandwidth design rules to pole-placement theory [2].
- Analyse closed-loop stability and disturbance rejection through characteristic polynomials and Lyapunov arguments as in Han [1].
- Apply the theory to a mass–spring–damper case study with multiple bandwidth selections, supported by reproducible Python tutorials.

## 10.1 Concept Map for Beginners

### Concept Map

1. View ADRC as “estimate the disturbance, then cancel it”—this story makes the extended state and observer gains feel natural.
2. Walk through the ESO pole placement slowly; it is just classical observer design with a different target polynomial.
3. Relate bandwidth choices to everyday terms: a faster ESO hears disturbances quickly, while controller bandwidth sets how aggressively you respond.

## 10.2 Concept–Math–Engineering Loop

## 10.3 Architecture Overview

### What?

Presents the three-block ADRC structure: tracking differentiator, extended state observer, and disturbance-compensating controller.

| Concept           | Mathematics                                    | Engineering Practice                                                                                      |
|-------------------|------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| ADRC architecture | Extended state definition, disturbance lumping | Sketch block diagrams labelling measured states, estimated disturbance, and compensation path.            |
| ESO design        | Pole placement via $(s + \omega_0)^3$          | Run <code>tutorial_adrc</code> to print <code>np.linalg.eigvals(A_o - L @ C)</code> and confirm gains.    |
| Bandwidth tuning  | Characteristic polynomials, gain scheduling    | Compare <code>figures/adrc_responses.png</code> across bandwidth sets to see predicted transient changes. |

Table 10.1: How Module 10 ties the ADRC concept to algebra and reproducible simulations.

### Why?

Understanding each block clarifies how ADRC achieves robustness without detailed plant models.

### When?

Before deriving equations or tuning bandwidths for a specific plant.

### How?

Express the plant with an extended state, design the ESO, and define the control law that cancels the disturbance estimate.

### Example

Draw the ADRC block diagram for a second-order plant and identify where each estimated state feeds back into the control input.

The ADRC philosophy treats unknown dynamics  $f(t)$  and disturbances as an extended state. A canonical second-order plant is written as

$$\ddot{y} = f(t, y, \dot{y}) + bu, \quad b > 0, \quad (10.1)$$

where  $b$  is the nominal control gain. ADRC comprises three blocks:

1. **Tracking Differentiator (TD)** (optional in LADRC): filters the reference signal and generates smooth derivatives.
2. **Extended State Observer (ESO)**: estimates  $y$ ,  $\dot{y}$ , and the lumped disturbance  $f$ .
3. **Nonlinear or linear feedback law**: cancels the disturbance estimate and imposes desired dynamics on the remaining subsystem.

This chapter focuses on the widely used linear ADRC variant where the TD is replaced by a simple second-order reference model.

## 10.4 Linear ADRC Derivation

### What?

Step-by-step derivation of the linearised ADRC equations for second-order systems.

**Why?**

Provides explicit formulas for ESO gains and feedback coefficients needed in software implementations.

**When?**

After introducing the architecture and before coding the controller.

**How?**

Form the extended state, place observer poles via  $(s + \omega_0)^3$ , and design the feedback law with desired closed-loop bandwidth  $\omega_c$ .

**Example**

Compute  $\beta_1 = 3\omega_0$ ,  $\beta_2 = 3\omega_0^2$ ,  $\beta_3 = \omega_0^3$  for  $\omega_0 = 200$  rad/s and verify numerically that `np.linalg.eigvals(A_o - L @ C)` returns all poles at  $-200$ .

Introduce the extended state  $x_e = [x_1 \ x_2 \ x_3]^T = [y \ \dot{y} \ f]^T$ . The ESO is designed such that the estimation error dynamics follow a third-order stable polynomial.

**10.4.1 Extended State Observer**

Define the observer

$$\dot{\hat{x}}_1 = \hat{x}_2 - \beta_1(\hat{x}_1 - y), \quad (10.2)$$

$$\dot{\hat{x}}_2 = \hat{x}_3 - \beta_2(\hat{x}_1 - y) + bu, \quad (10.3)$$

$$\dot{\hat{x}}_3 = -\beta_3(\hat{x}_1 - y), \quad (10.4)$$

with gains

$$\beta_1 = 3\omega_0, \quad \beta_2 = 3\omega_0^2, \quad \beta_3 = \omega_0^3, \quad (10.5)$$

where  $\omega_0$  is the observer bandwidth. The pole placement is explicit: the error dynamics matrix is

$$A_o - LC = \begin{bmatrix} -\beta_1 & 1 & 0 \\ -\beta_2 & 0 & 1 \\ -\beta_3 & 0 & 0 \end{bmatrix},$$

whose characteristic polynomial is

$$\det(sI - (A_o - LC)) = s^3 + \beta_1 s^2 + \beta_2 s + \beta_3.$$

Setting this polynomial equal to  $(s + \omega_0)^3 = s^3 + 3\omega_0 s^2 + 3\omega_0^2 s + \omega_0^3$  yields the gain triplet in (10.5). Insert a diagnostic print in `tutorial_adrc` to evaluate `np.linalg.eigvals(A_o - L @ C)` and confirm numerically that all poles equal  $-\omega_0$ . Substituting (10.2)–(10.4) with these gains reveals that the error dynamics satisfy

$$e^{(3)} + 3\omega_0 e^{(2)} + 3\omega_0^2 e^{(1)} + \omega_0^3 e = 0, \quad (10.6)$$

analogous to placing the observer poles at  $-\omega_0$ . In discrete implementation we use Euler integration in the tutorial to remain consistent with control-loop timing.

### 10.4.2 Control Law and Closed-Loop Poles

The compensated control input is

$$u = \frac{v - \hat{x}_3}{b}, \quad v = k_p(r - \hat{x}_1) - k_d\hat{x}_2, \quad (10.7)$$

with feedback gains chosen according to the desired closed-loop polynomial

$$s^2 + 2\omega_c s + \omega_c^2 = 0, \quad k_p = \omega_c^2, \quad k_d = 2\omega_c. \quad (10.8)$$

Combining (10.1) with (10.7) yields the characteristic polynomial

$$s^2 + 2\omega_c s + \omega_c^2 = 0, \quad (10.9)$$

indicating that perfect disturbance cancellation reduces the plant to a double integrator with bandwidth  $\omega_c$ . In practice  $\hat{x}_3$  approximates  $f$ , so robustness hinges on the ESO's ability to track  $f$  faster than the closed-loop dynamics (typically  $\omega_0 \geq 3\omega_c$ ).

### 10.4.3 Code Mapping

| Python                           | Formula                                                            | Explanation                                                                              |
|----------------------------------|--------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| <code>build_eso()</code>         | $\beta_1 = 3\omega_0, \beta_2 = 3\omega_0^2, \beta_3 = \omega_0^3$ | Utility in <code>tutorials.adrc</code> that generates ESO gains from the bandwidth rule. |
| <code>design_controller()</code> | $k_p = \omega_c^2, k_d = 2\omega_c, u = (v - \hat{x}_3)/b$         | Implements the linear ADRC control law once $\omega_c$ and $b$ are specified.            |

Table 10.2: Mapping LADRC formulas to the tutorial code.

## 10.5 Stability and Disturbance Rejection

### What?

Analyses how ESO accuracy and bandwidth choices influence closed-loop stability and disturbance suppression.

### Why?

Ensures ADRC configurations remain stable when disturbances or modelling errors occur.

### When?

After deriving the control law to check robustness before hardware deployment.

### How?

Examine characteristic polynomials, Lyapunov functions, and frequency-domain interpretations.

### Example

Show that choosing  $\omega_0 \geq 3\omega_c$  maintains small estimation error and plot residual disturbances using the tutorial scripts.

### 10.5.1 Error Dynamics

Define the estimation error  $\tilde{x} = x_e - \hat{x}$ . From (10.2)–(10.4) the error dynamics are

$$\dot{\tilde{x}} = (A_o - LC)\tilde{x} + [0 \ 0 \ \dot{f}]^T, \quad (10.10)$$

with

$$A_o = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}, \quad L = [\beta_1 \ \beta_2 \ \beta_3]^T, \quad C = [1 \ 0 \ 0]. \quad (10.11)$$

The homogeneous part has eigenvalues at  $-\omega_0$  (triple pole). If  $f$  is slowly varying,  $\dot{f}$  acts as a small perturbation, yielding bounded estimation error. This motivates the tuning guideline  $\omega_0 \gg$  dominant frequencies of  $f$ .

### 10.5.2 Lyapunov Interpretation

Following Han [1], choose the Lyapunov function  $V = \tilde{x}^T P \tilde{x}$  where  $P$  solves the Lyapunov equation

$$(A_o - LC)^T P + P(A_o - LC) = -Q, \quad (10.12)$$

for  $Q \succ 0$ . With the gain selection (10.5),  $A_o - LC$  is Hurwitz for any  $\omega_0 > 0$ , guaranteeing exponential convergence of  $\tilde{x}$  in the absence of disturbance derivatives. The controller in (10.7) then renders the closed-loop error dynamics

$$\ddot{e} + 2\omega_c \dot{e} + \omega_c^2 e = \tilde{x}_3, \quad (10.13)$$

illustrating that disturbances are attenuated according to the ESO error.

## 10.6 Tuning Workflow

### What?

Practical steps for selecting bandwidths, controller gains, and noise filters in ADRC.

### Why?

A disciplined tuning procedure accelerates deployment and avoids guesswork.

### When?

When adapting ADRC to a new plant or retuning after requirement changes.

### How?

Pick target  $\omega_c$ , set  $\omega_0$ , adjust gain scaling, and iterate with simulation feedback.

### Example

Follow the provided table of slow/medium/fast bandwidths and evaluate resulting step responses to choose a balanced configuration.

1. Choose the tracking bandwidth  $\omega_c$  based on transient requirements (settling time roughly  $4/\omega_c$ ).
2. Set the observer bandwidth  $\omega_0$  between three and five times  $\omega_c$  to ensure fast disturbance estimation without amplifying measurement noise [2].
3. Validate the design via simulation, examining both tracking and control effort.

4. If large measurement noise is present, reduce  $\omega_0$  and consider filtering the measured output.
5. For plants with uncertain  $b$ , identify or adaptively estimate the gain using techniques from Chapter 9.

## 10.7 Case Study: Mass–Spring–Damper

### What?

Applies ADRC formulas to a familiar second-order plant with quantifiable performance.

### Why?

Demonstrates how tuning selections translate into measurable overshoot, settling time, and control effort.

### When?

After learning the general workflow—use the case as a reference implementation.

### How?

Implement ESO and controller gains, impose actuator limits, and compare bandwidth configurations.

### Example

Run `python -m scripts.tutorials adrc --output-dir figures`  
and inspect `figures/adrc_responses.png` to compare slow/medium/fast designs.

For  $m = 1$ ,  $c = 0.4$ ,  $k = 4$  the nominal input gain is  $b = 1$ . We compare three bandwidth selections:

| Scenario | $\omega_c$ | $\omega_0$ |
|----------|------------|------------|
| Slow     | 40         | 120        |
| Medium   | 80         | 200        |
| Fast     | 120        | 300        |

The simulations include actuator saturation at  $\pm 7$  N to highlight practical limitations. Steady-state error and percent overshoot for each case are printed by the tutorial script.

## 10.8 Tutorial: ADRC Walkthrough

### What?

Script-driven exploration of ADRC responses, control effort, and observer errors.

### Why?

Visual feedback solidifies how bandwidth choices affect tracking and disturbance rejection.

### When?

After derivations, during tuning, or when preparing documentation.

### How?

Execute the tutorial, generate response/control/ESO plots, and adjust parameters to see their impact.

**Example**

Modify  $\omega_c$  in the script, regenerate plots, and note how control saturation increases for aggressive bandwidths.

1. Run `python -m scripts.tutorials adrc --output-dir figures`.
2. Analyse Figure 10.1 to see how bandwidth choice affects transient metrics.
3. Examine Figure 10.2 to understand the variation in control effort and saturation utilisation.
4. Study Figure 10.3 to evaluate ESO estimation error and confirm convergence consistent with the Lyapunov discussion above.
5. Modify parameters in `tutorial_adrc` to explore robustness against plant parameter mismatch and measurement noise.

See Appendix B for the standard command palette that recreates these ADRC experiments and keeps figures in sync with the notes.

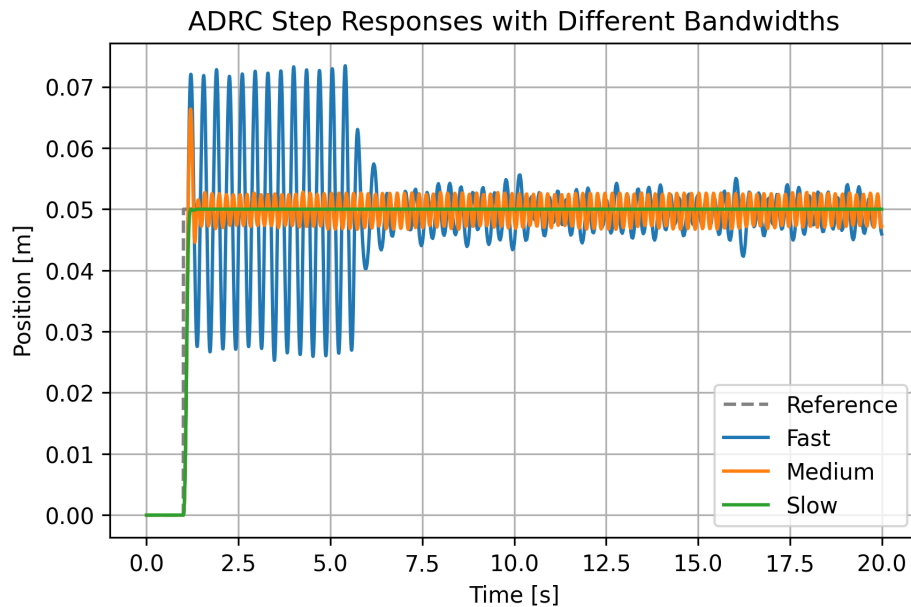


Figure 10.1: ADRC step responses for multiple bandwidth selections.

## 10.9 Discussion and Further Reading

**What?**

Highlights extensions, research directions, and connections to broader ADRC literature.

**Why?**

Guides further study and contextualises the linear ADRC variant within the wider field.

**When?**

After mastering the basics and seeking advanced topics such as MIMO or nonlinear ADRC.

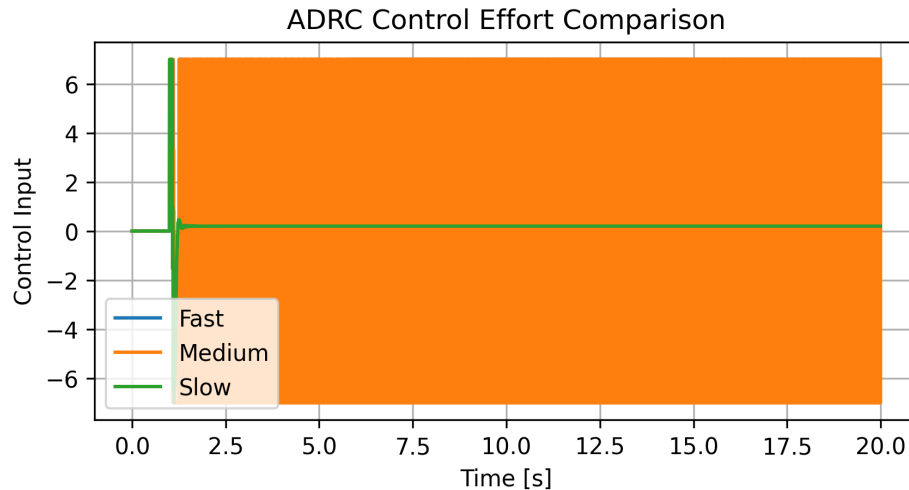


Figure 10.2: Control effort comparison across ADRC bandwidths.

### How?

Review recommended papers, compare methodologies, and identify scenarios where advanced observers add value.

### Example

Read Gao [2] and replicate their higher-order ESO design using the provided code scaffold.

The LADRC design summarised here linearises the broader nonlinear ADRC methodology in Han [1]. Extensions include:

- Higher-order plants and multivariable systems using generalised ESOs [2].
- Integration with learning-based gain identification (Chapter 9) to adapt to changing plant gains.
- Nonlinear ADRC variants where the observer uses saturation functions to improve robustness to noise.

### Key Takeaways

- ADRC delivers disturbance rejection without detailed plant models, making it attractive to beginners once the ESO concept is internalised.
- Formal analysis shows that bandwidth choices correspond to explicit pole locations, providing a concrete tuning recipe.
- Simulation and tutorial scripts bridge the gap between closed-form derivations and practical implementation, preparing learners for advanced ADRC literature.

## 10.10 Module Review

- **Architecture:** separate reference smoothing, extended state observation, and disturbance-compensating feedback.



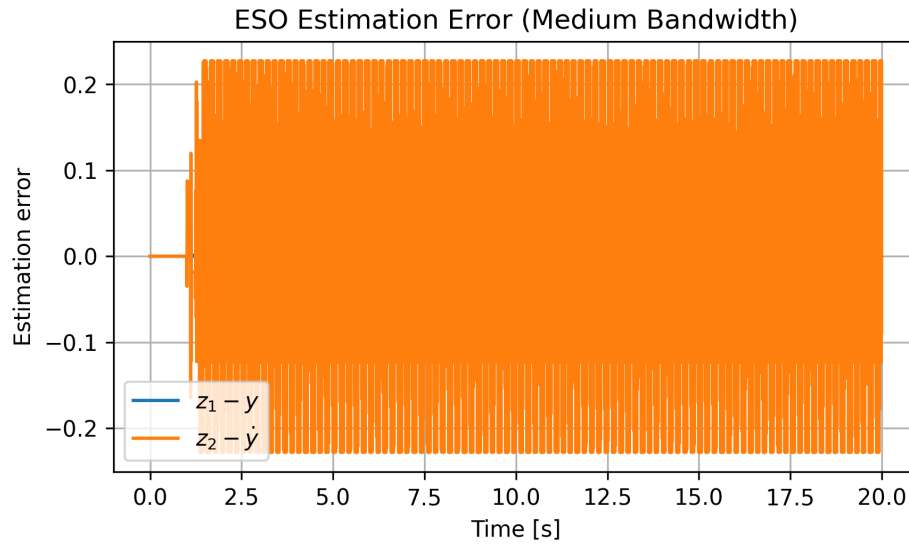


Figure 10.3: ESO estimation error for the medium-bandwidth design.

- **ESO gains:**  $\beta_1 = 3\omega_0$ ,  $\beta_2 = 3\omega_0^2$ ,  $\beta_3 = \omega_0^3$  place observer poles at  $-\omega_0$ .
- **Control law:**  $u = (v - \hat{x}_3)/b$  with  $k_p = \omega_c^2$ ,  $k_d = 2\omega_c$  imposes desired second-order dynamics.
- **Bandwidth heuristic:** choose  $\omega_0 \geq 3\omega_c$  so disturbance estimates keep pace with the controller.
- **Practice:** run `python -m scripts.tutorials adrc` and vary  $(\omega_c, \omega_0)$  to observe trade-offs in Figures 10.1–10.3.

## 10.11 Keyword Review

### Keywords

- **ADRC:** Active Disturbance Rejection Control; framework that estimates and cancels disturbances via an extended state.
- **Extended State Observer (ESO):** observer estimating plant state and lumped disturbance with gains tied to  $\omega_0$ .
- **Bandwidth**  $\omega_c$ ,  $\omega_0$ : controller and observer speed parameters guiding transient response and estimation speed.
- **Disturbance estimate**  $\hat{x}_3$ : online approximation of unknown dynamics subtracted from the input.
- **Tracking differentiator:** optional reference filter that provides smoothed derivatives for ADRC.



# Appendix A

## Mathematical Background

### A.1 Linear Algebra Reference

#### A.1.1 Eigenvalues and Modal Decomposition

For  $A \in \mathbb{R}^{n \times n}$  with distinct eigenvalues, the modal decomposition  $A = V\Lambda V^{-1}$  expresses solutions of  $\dot{x} = Ax$  as

$$x(t) = Ve^{\Lambda t}V^{-1}x(0) = \sum_{i=1}^n e^{\lambda_i t} v_i c_i, \quad (\text{A.1})$$

where  $v_i$  are eigenvectors and  $c_i$  scalar weights. Complex conjugate eigenvalues form oscillatory modes.

#### A.1.2 Matrix Norms and Conditioning

The induced 2-norm  $\|A\|_2$  equals the largest singular value of  $A$ . Condition numbers obey  $\kappa_2(A) = \sigma_{\max}(A)/\sigma_{\min}(A)$  and measure sensitivity of linear solves. When  $\kappa_2(A)$  is large, numerical errors can significantly distort state-space computations.

#### A.1.3 Pseudoinverse

For non-square matrices, the Moore–Penrose pseudoinverse  $A^+ = V\Sigma^+U^T$  (from SVD  $A = U\Sigma V^T$ ) yields minimum-norm least-squares solutions to  $Ax = b$  via  $x = A^+b$ .

### A.2 Differential Equations

#### A.2.1 Variation of Parameters

For  $\dot{x} = Ax + Bu$ , define the transition matrix  $\Phi(t, t_0) = e^{A(t-t_0)}$ . The solution with time-varying input is

$$x(t) = \Phi(t, t_0)x(t_0) + \int_{t_0}^t \Phi(t, \tau)Bu(\tau) d\tau. \quad (\text{A.2})$$

Discrete-time analog:  $x_{k+1} = A_dx_k + B_d u_k$  with  $A_d = e^{AT_s}$  and  $B_d$  defined similarly.

### A.2.2 Riccati Equations

The discrete algebraic Riccati equation (DARE) appears in LQR and Kalman filtering:

$$P = A^T P A - A^T P B (B^T P B + R)^{-1} B^T P A + Q. \quad (\text{A.3})$$

Solution existence requires  $(A, B)$  stabilizable and  $(A, Q^{1/2})$  detectable.

## A.3 Laplace Transform Reference

For a causal signal  $x(t)$ ,

$$X(s) = \mathcal{L}\{x(t)\} = \int_0^\infty x(t)e^{-st} dt.$$

Table A.1 lists representative pairs used throughout the notes. Key properties: differentiation

| Time signal       | Transform                 | Notes           |
|-------------------|---------------------------|-----------------|
| 1                 | $1/s$                     | Step input      |
| $t$               | $1/s^2$                   | Ramp input      |
| $e^{-at}$         | $1/(s+a)$                 | Delay property  |
| $\sin(\omega t)$  | $\omega/(s^2 + \omega^2)$ | Imaginary poles |
| $\cos(\omega t)$  | $s/(s^2 + \omega^2)$      | Even symmetry   |
| $\delta(t - t_0)$ | $e^{-st_0}$               | Shifted impulse |

Table A.1: Selected Laplace transform pairs.

$\Rightarrow \mathcal{L}\{\dot{x}\} = sX(s) - x(0)$ , integration  $\Rightarrow \mathcal{L}\{\int_0^t x\} = X(s)/s$ , and convolution  $\Rightarrow \mathcal{L}\{x*y\} = X(s)Y(s)$ .

## A.4 $z$ -Transform Reference

For a sequence  $x[k]$ ,

$$X(z) = \mathcal{Z}\{x[k]\} = \sum_{k=0}^{\infty} x[k]z^{-k}.$$

Table A.2 summarises canonical pairs along with their regions of convergence (ROC). Useful iden-

| Sequence          | Transform                 | ROC            | Notes                |
|-------------------|---------------------------|----------------|----------------------|
| 1                 | $1/(1 - z^{-1})$          | $ z  > 1$      | Unit step            |
| $k$               | $z^{-1}/(1 - z^{-1})^2$   | $ z  > 1$      | Discrete ramp        |
| $a^k$             | $1/(1 - az^{-1})$         | $ z  >  a $    | Exponential mode     |
| $ka^k$            | $az^{-1}/(1 - az^{-1})^2$ | $ z  >  a $    | Weighted exponential |
| $\delta[k - k_0]$ | $z^{-k_0}$                | All $z \neq 0$ | Shifted unit sample  |

Table A.2: Selected  $z$ -transform pairs.

ties: time shift  $\Rightarrow \mathcal{Z}\{x[k - k_0]\} = z^{-k_0}X(z)$ , forward shift  $\Rightarrow \mathcal{Z}\{x[k + 1]\} = zX(z) - x[0]$ , and convolution  $\Rightarrow \mathcal{Z}\{x * y\} = X(z)Y(z)$ . The discretisation link  $z = \exp(sT_s)$  connects continuous poles with their sampled counterparts as described in Chapter 2.

## A.5 Probability Primer

### A.5.1 Gaussian Random Vectors

If  $x \sim \mathcal{N}(\mu, \Sigma)$  and  $y = Cx + d$ , then  $y$  is Gaussian with mean  $C\mu + d$  and covariance  $C\Sigma C^T$ . The conditional distribution of  $x$  given  $y$  is also Gaussian, forming the basis for Kalman filtering.

### A.5.2 Law of Total Expectation and Variance

For random variables  $X$  and  $Y$ ,

$$\mathbb{E}[X] = \mathbb{E}[\mathbb{E}[X | Y]], \quad (\text{A.4})$$

$$\text{Var}(X) = \mathbb{E}[\text{Var}(X | Y)] + \text{Var}(\mathbb{E}[X | Y]). \quad (\text{A.5})$$

These identities often appear when analyzing estimation error dynamics.

## A.6 Scalar Kalman Filter Recap

For the scalar process

$$x_k = x_{k-1} + w_k, \quad z_k = x_k + v_k,$$

with  $w_k \sim \mathcal{N}(0, Q)$  and  $v_k \sim \mathcal{N}(0, R)$ , the prediction/update equations simplify to

$$\begin{aligned} \hat{x}_{k|k-1} &= \hat{x}_{k-1|k-1}, & P_{k|k-1} &= P_{k-1|k-1} + Q, \\ K_k &= \frac{P_{k|k-1}}{P_{k|k-1} + R}, & \hat{x}_{k|k} &= \hat{x}_{k|k-1} + K_k(z_k - \hat{x}_{k|k-1}), \\ P_{k|k} &= (1 - K_k)P_{k|k-1}. \end{aligned}$$

The deduction mirrors the workflow in the standalone reference `kalman_deduction.tex`: take expectations of the process equation for the prediction step, compute the innovation  $y_k = z_k - \hat{x}_{k|k-1}$ , and minimise the posterior variance with respect to  $K_k$ . This scalar case provides intuition before tackling the matrix equations in Chapter 6.



# Appendix B

## Tooling and Workflow

### B.1 Repository Layout

- `tex/`: LaTeX sources organized by module; compile with Xe<sub>La</sub>TeX via the `Makefile`.
- `scripts/`: reusable Python modules (dynamics, controllers, simulation, visualization, experiments).
- `notebooks/`: optional Jupyter notebooks for symbolic or exploratory analysis.
- `build/`: generated artifacts such as `course.pdf` and saved figures.

### B.2 Compiling the Notes

1. Confirm Xe<sub>La</sub>TeX is available. Under WSL, invoke the Windows installation with

```
make XELATEX=xelatex.exe BIBER=biber.exe
```

2. Run `make`; the workflow enters `tex/` before compiling so local inputs are resolved.
3. Use `make clean` to clear auxiliary files when necessary.

### B.3 Python Environment

1. Activate `.venv` and install dependencies listed in `requirements.txt`.
2. Execute experiments: `python demo.py <experiment> [--no-plot --output path]`.
3. For repeatable plots, save figures to `figures/` and include them in the LaTeX sources via `\includegraphics`.

### B.4 Automation Hooks

- Extend the `Makefile` with targets such as `make figures` that call Python scripts and copy generated PNG files into `build/figures`.
- Integrate static analysis tools (e.g., `ruff`, `black`) as the code base grows; add targets to run unit tests once created.

- Record new workflow steps in `agents.md` to keep all collaborators aligned.



# References



# Bibliography

- [1] J. Han, “Auto-disturbance-rejection control technique: Advances and applications,” *Frontiers of Electrical and Electronic Engineering in China*, vol. 4, no. 3, pp. 273–279, 2009.
- [2] Z. Gao, “Active disturbance rejection control: A paradigm shift in feedback control system design,” *Proceedings of the American Control Conference*, pp. 2399–2405, 2006.
- [3] K. J. Åström and T. Hägglund, *PID Controllers: Theory, Design, and Tuning*, 2nd ed. ISA, 1995.
- [4] K. J. Åström and T. Hägglund, *Advanced PID Control*. ISA, 2006.
- [5] L. Ljung, *Control System Design*. Studentlitteratur, 2012.
- [6] M. Krstić, I. Kanellakopoulos, and P. V. Kokotović, *Nonlinear and Adaptive Control Design*. Wiley, 1995.